



单位代码： 10635

西南大学

本科毕业论文（设计）

基于大模型的数据库查询系统设计与实现

学 院： 计算机与信息科学学院 软件学院

专 业： 软件工程（中外合作办学）

姓 名： 李牧筱

学 号： 222022321062067

指导教师： 詹成

答辩日期： 2026 年 月 日

2026 年 4 月 9 日

独 创 声 明

本人郑重声明：所呈交的毕业论文（设计），是本人在指导老师的指导下，独立进行研究工作所取得的成果，成果不存在知识产权争议。尽我所知，除文中已经注明引用的内容外，本论文（设计）不含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体均已在文中以明确方式标明。

声明的法律后果由本人承担。

作者签名：_____李牧筱_____

2026 年 4 月 9 日

毕业论文（设计）使用授权声明

本人完全了解西南大学关于收集、保存、使用毕业论文（设计）的规定。

本人愿意按照学校要求提交论文（设计）的印刷本和电子版，同意学校保存论文（设计）的印刷本和电子版，或采用影印、数字化或其它复制手段保存论文（设计）；同意学校在不以营利为目的的前提下，建立目录检索与阅览服务系统，公布论文（设计）的部分或全部内容，允许他人依法合理使用。

（保密论文在解密后遵守此规定）

作者签名：_____李牧筱_____

2026 年 4 月 9 日

目录

摘 要	1
Abstract	2
第 1 章 导论	3
1.1 研究背景及意义	3
1.1.1 大语言模型智能体的应用趋势	4
1.1.2 大语言模型智能体的困境	4
1.1.3 自评估模型构建的必要性	4
1.2 国内外研究现状	5
1.2.1 Text-to-SQL 技术演进	5
1.2.2 基于大模型的智能代理 (Agent) 架构	5
1.3 主要研究内容与目标	6
1.4 论文组织结构	7
第 2 章 相关理论与技术基础	8
2.1 自然语言处理基础	8
2.1.1 自然语言处理技术的历史演进	8
2.1.2 词嵌入技术在自然语言处理中的应用	9
2.1.3 提示词工程	9
2.2 Transformer 与 BERT 基础	10
2.2.1 Transformer 的历史与架构突破	10
2.2.2 Transformer 的核心数学机理	10
2.2.3 从 Transformer 到 BERT 的演进	12
2.2.4 BERT 的输入表示与特征浓缩架构	12
2.3 数据库理论基础	13
2.3.1 关系数据库基本概念	13
2.3.2 MySQL 架构设计	14
2.3.3 向量数据库基本概念	15
2.4 智能体架构理论与设计	15
2.4.1 环境感知与结构知识索引	16

2.4.2	基于反馈的动作执行与自修复闭环	16
2.4.3	智能体状态自评估与动态并发策略	17
2.5	本章小结	18
第 3 章	基于 Agent 的 Text-to-Python 代码生成与自修复机制研究	19
3.1	Text-to-Python 代理决策框架设计	19
3.2	动态 Schema 感知与 DataFrame 映射	19
3.2.1	MySQL 物理模型到 DataFrame 的内存级映射	19
3.2.2	基于 PGVector 的结构知识语义召回	21
3.3	Text-to-Python 代码生成架构设计	22
3.3.1	任务降维：从声明式 SQL 到命令式 Pandas 的范式转换	22
3.3.2	提示词模板工程与动态上下文组装	22
3.3.3	基于安全沙箱的隔离执行机制	25
3.4	基于异常回输的代码自修复闭环机制	33
3.4.1	Agent 自修复核心轮询逻辑	33
3.4.2	针对 Pandas 运行时异常的语境重构	35
3.5	多代理并发调度与早停机制	36
第 4 章	基于 BERT 的查询并发预测与调度模型设计	41
4.1	存在问题	41
4.2	研究动机	41
4.3	并发预测模型设计	42
4.3.1	向量化特征提取机制	43
4.3.2	并发度回归层设计与目标函数	44
4.4	动态调度策略实现	45
4.5	实验设计与分析	45
4.5.1	实验环境与数据集构建	46
4.5.2	并发度与任务成功率的边际效用分析	46
4.5.3	预测调度在延迟开销上的置换效益评估	47
4.6	本章小结	48
第 5 章	数据库高效查询原型系统设计与实现	49
5.1	系统需求分析	49

5.1.1	系统角色分析	49
5.1.2	功能性需求与成文用例分析	49
5.1.3	非功能性需求	55
5.2	系统架构设计	57
5.2.1	总体微服务架构拓扑	57
5.2.2	核心模块动态交互时序	59
5.2.3	数据库表结构图	60
5.2.4	数据模型设计	62
5.2.5	PGVector 结构知识库设计	65
5.3	系统实现与展示	66
5.3.1	身份认证与访问控制实现	66
5.3.2	自然语言交互与预处理调度引擎	67
5.3.3	隔离沙箱执行与结果渲染展示	68
5.4	系统测试与分析	69
5.4.1	测试环境配置	70
5.4.2	核心功能与安全测试用例	70
5.4.3	综合测试分析	72
5.5	本章小结	72
第 6 章	结论与展望	73
6.1	全文总结	73
6.2	研究不足与未来工作	74
6.2.1	对隐式业务逻辑与复杂时序计算的支持局限	74
6.2.2	并发预测模型的跨领域泛化与自适应演进	74
6.2.3	底层基座模型的领域专属微调优化	74
	参考文献	76
	致 谢	78

基于大模型的数据库查询系统设计与实现

李牧筱

西南大学计算机与信息科学学院 软件学院, 重庆 400715

摘 要：针对传统 Text-to-SQL 方法在实际使用中容易生成错误逻辑、存在安全隐患以及浪费计算资源等问题，本文设计并实现了一套基于大语言模型的数据库智能查询系统。系统整体采用了前后端分离与微服务架构。前端基于 Vue 框架，主要负责页面的交互与结果的可视化展示；后端基于 Django 框架，通过让大模型生成 Python 代码来替代直接生成 SQL 语句。结合 PGVector 表结构检索和代码沙箱环境，系统能够截获代码运行时的报错信息，并引导大模型自动进行修改和重试。此外，系统将算法评估部分拆分为独立的微服务，利用 BERT 模型来预测查询任务的难度，从而动态分配合适的并发线程数。最后的测试结果表明，该系统在保证查询准确率和底层数据库安全的前提下，有效减少了不必要的接口调用开销，为日常的数据交互提供了一种稳定、实用的解决思路。

关键词：Text-to-Python；大语言模型；代码自修复；并发预测；动态调度

Design and Implementation of Database Query System Based on Large Language Models

LI Muxiao

College of Computer and Information Science & School of Software, Southwest University, Chongqing

400715, PR China

Abstract: Traditional Text-to-SQL methods often generate incorrect logic, pose security risks, and waste computing resources in practical applications. To solve these problems, this thesis designs and implements an intelligent database query system based on large language models. The system adopts a frontend-backend separation and microservice architecture. The frontend, based on the Vue framework, is mainly responsible for user interaction and data visualization. The backend, based on the Django framework, uses a Text-to-Python approach instead of generating SQL statements directly. By combining PGVector for database schema retrieval and a code sandbox environment, the system can capture execution errors and guide the large language model to automatically modify and retry the code. In addition, the system separates the algorithm evaluation part into an independent microservice, using a BERT model to predict the difficulty of the query task and dynamically allocate the appropriate number of concurrent threads. Test results show that the system can effectively reduce unnecessary API calling costs while ensuring query accuracy and underlying database security, providing a stable and practical solution for daily data interaction.

Key words: Text-to-Python; Large Language Models; Code Self-Correction; Concurrency Prediction; Dynamic Scheduling

第 1 章 导论

1.1 研究背景及意义

在数字化转型逐步深入的时代，数据已成为企业中最核心的生产要素之一，虽然关系型数据库已为其奠定了良好的数据底座，但“数据鸿沟”的问题依然较为明显，业务决策者迫切需要实时、灵活的数据洞察，然而较高的 SQL 查询工具门槛却使得企业中没有技术背景的人员无法使用复杂的报表开发流程，随着大语言模型（LLM）的发展，Text-to-SQL 技术迎来了高速发展，但在实际的工业级生产环境下，简单的“自然语言进，SQL 语句出”的模式却遭遇了更大的挑战，首先是执行安全与表达能力的隐患，生成直接的 SQL 语句存在直接的注入威胁，同时对于逻辑复杂的统计计算，其灵活性不及高级编程语言；其次是大模型的“幻觉”问题：模型本身是概率预测的问题，所生成的逻辑极易发生大量、细微但致命的错误，且不具备有效的自我纠错能力；最后是算力成本与响应时延的矛盾，盲目增加并发调用将导致巨大的 API 费用开销。针对上述问题的痛点，本课题组开发了一套基于 Vue + Django + 独立算法模块的微服务系统，转向更具普适性的 Text-to-Python 模式，通过建立自我修复能力的智能代理 Agent 结合 BERT 模型进行复杂度感知下的资源调度。

本课题开展的研究工作，不仅在理论层面上有很多探索的价值，在真正的工程实践中也有着突出的现实意义。第一点是在理论研究这方面，本课题专门钻研了如何把程序运行时的报错异常转化成大模型能识别的反馈信号，由此把“生成、执行、反馈、修复”这种闭环的认知模型给构建了出来，这也就给智能代理去处理复杂任务时的自主进化提供了一套新思路；第二点则是把 BERT 模型给引入了进来，针对自然语言的意图去做了向量化的深度分析，还顺带把一套基于联合概率的并发推导模型给建立好了，给大模型应用里的资源管理和成本控制提供了有力的数学层面的支持，从而形成了这种“大模型 + 小模型”协同调度的创新应用模式。至于在工程实践与具体的应用价值上，本系统是把前后端分离的架构和代码沙箱的处理给结合到了一起，在把数据库安全给保障好的基础上，让那些没学过编程的非专业人员，只用自然语言的口语指令就能完成专业化的数据分析，从而有力地促进了数据库的使用便利性。

系统所采用的动态并发策略能够有效地平衡生成成功率与 API 调用成本两者之间的矛盾，使得本方案可在初创企业看板和医疗数据检索等实际场景下得以高效地落地、实施。不仅如此，系统通过 Django 异步网关的设计与高性能连接池的

管理，还表明了如何将先进的 AI 代理更加平滑地集成到传统的信息技术（IT）架构中，推进企业的信息管理系统向智能化、自适应化方向升级。综上所述，本研究的目的不仅仅在于根据已实现的数据库库表设计解决数据库查询效率的问题，更在于对于大模型工程化落地模式提供一条稳定、安全、廉价的实现途径进行探索，为下一代智能数据交互工具的实现提供有益的参考。

1.1.1 大语言模型智能体的应用趋势

随着以 GPT 系列为代表的大语言模型在超大规模参数预训练的技术下展示出颠覆性的进步，人工智能由单纯的文本生成工具逐渐演进为具备逻辑推理能力和任务解析能力的智能体（Agent）。在现代的工业界，这些智能体不仅能够完成代码调试、客户服务等专业化任务，更在多智能体（multi-agent）的协作框架下展现了优化复杂供应链和金融风险评估等方面潜力的应用。与传统的基于规则的系统相比，大语言模型的智能体凭借自然语言的交互，能够自主解析任务目标完成复杂决策链（chain of decision）的执行，而这正是企业智能化转型的重要基础设施。

1.1.2 大语言模型智能体的困境

尽管大语言模型表现出了卓越的认知能力，但是其固有的技术缺陷也在很大程度上制约了大语言模型在极其严苛的生产环境之中的实际应用价值。在开放域问题处理上，顶级模型依然不能完全躲避“幻觉”现象的出现，即模型在处理过程中生成的不切实际或逻辑不符的代码与结论形式给其可靠性带来了深深的质疑。除此之外，应用模型的响应质量具有明显的波动性，极度依赖于提示词的预设设计和上下文窗口（context window）的长度的限制。长文本的处理和在复杂模式映射（Schema linking）场合下，模型极为容易忘记重要的前置信息，造成执行的效率急剧下降。加之大模型内部工作机制的“黑盒”特性，开发者往往难以解释其特定决策过程，也无法对其生成输出实现精准控制。

1.1.3 自评估模型构建的必要性

针对大模型普遍存在的不确定性，如果只依赖单个智能体，很难满足高可靠性场景的要求。为了提高系统的稳定性，开发者往往会采用多个智能体并发运行的方法，但这又带来了资源浪费和成本上升的问题。因此，建立一种能动态感知的自评估机制就变得很关键。通过构建复杂度预测模型来预判任务成功的概率，系统可以根据任务的难易程度来动态调整并发线程的数量，实现资源的合理分配。本

研究尝试将 BERT 文本回归技术用在智能体的能力评估上，通过对用户提问进行语义分析，让 AI 系统具备自我认知和资源调度的能力，为构建稳定、廉价且高效的数据库查询工具提供了一种新的尝试。

1.2 国内外研究现状

1.2.1 Text-to-SQL 技术演进

Text-to-SQL 技术是连接自然语言和结构化数据库的关键，它的发展过程是随着自然语言处理和数据库技术的进步而不断演进的。整体来看，Text-to-SQL 的演进经历了四个阶段。一是基于规则和模板匹配的初始阶段，主要是通过人工预设 SQL 模板和关键词匹配来实现转换。这种方法虽然能处理简单的查询，但人工维护成本太高，也没法应对多表关联等复杂场景。二是基于神经网络的序列生成阶段，研究者把编码器-解码器架构引进来，结合注意力机制解决了 SQL 生成的问题，但在跨数据库使用时泛化能力比较差。三是模式链接与图神经网络融合阶段，这一阶段把数据库表结构信息引进了模型，通过建模表和字段的关系，提升了复杂查询的准确率，但推理延迟较高。四是目前由预训练大模型主导的智能化阶段，依靠大模型极强的语义理解能力，结合提示工程和检索增强技术，实现了在少样本场景下的适配。比如余洋在研究中引入了 LoRA 和 QLoRA 等微调技术，在降低计算资源消耗的同时，提升了 SQL 生成的准确性，推动了技术在企业中的落地^[1]。目前，基于预训练语言模型的方法已经成了主流^[2]，研究重点也从简单的语法匹配转向了复杂的模式对齐以及如何防范逻辑幻觉。

1.2.2 基于大模型的智能代理（Agent）架构

随着大模型能力的提升，利用它来构建智能代理（Agent）已成为解决复杂工程任务的一种共识。大模型 Agent 通常被认为具备感知环境、理解意图、制定决策和执行行动的能力，它一般由规划、记忆和工具使用这三个关键部分组成。

同时，大模型 Agent 的业务活动框架可以分为四个层次。首先是基础设施层，这是 Agent 运行的环境，涉及到服务器、权限、网络和数据库等，保证了 Agent 的正常运行。其次是数据层，它涵盖了数据的采集和预处理，任务的完成过程其实就是数据在其中的流转。然后是服务层，包括了底层的模型和所用到的工具；在本项目里，底层大模型用的是阿里巴巴的 Qwen 千问系列，工具则包括 MySQL 和 PGVector 等。最后是应用层，它整合了前面三个层次，直接面向用户输出处理结

果^[3]。不过，现在的很多 Text-to-SQL Agent 往往只关注单次查询的准确率，对于真实业务中并发量和 API 成本之间的平衡研究还比较少。

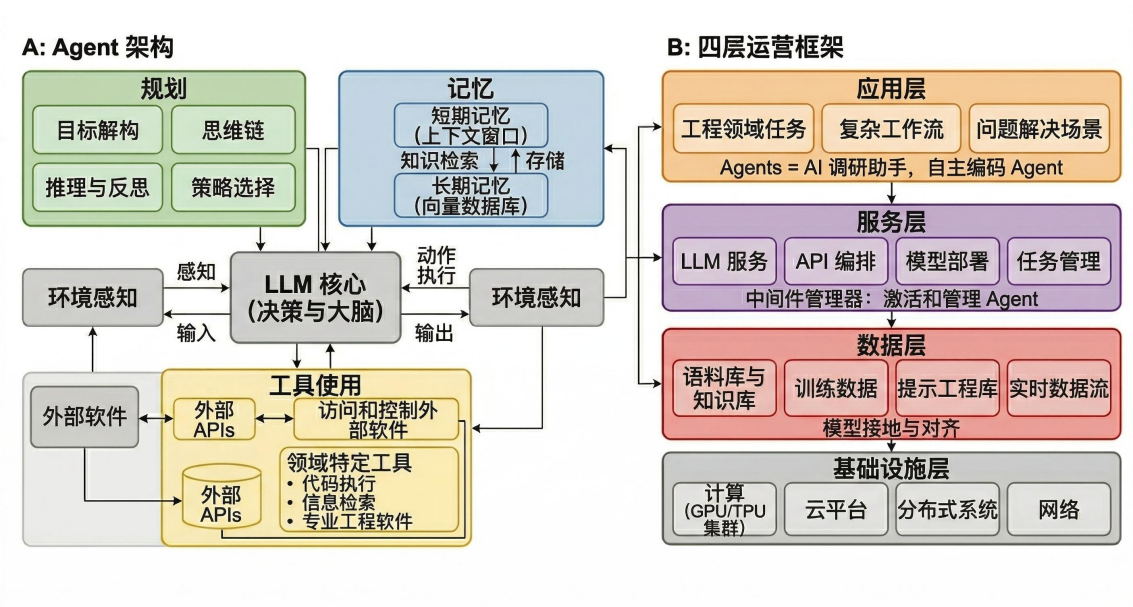


图 1.1 基于大语言模型的智能代理的架构与功能模型
Fig. 1.1 Architecture and Functional Model for LLM-Based Intelligent Agents

1.3 主要研究内容与目标

本论文主要关注大语言模型在数据库查询任务中容易产生幻觉、不确定性高以及盲目并发导致算力浪费等问题。为了提高系统的可靠性并控制成本，本课题设计并实现了一套具备自动纠错和动态并发调度能力的查询系统。主要的研究内容包括下面三个方面：

1. **基于闭环反馈的 Text-to-Python 智能体架构设计**：针对传统 Text-to-SQL 表达能力受限和安全风险的问题，设计一种以 Pandas DataFrame 为中间操作介质的代理架构。研究并实现基于 PGVector 的动态 Schema 感知技术，通过检索相关的表结构来压缩上下文。同时，构建一个安全的代码执行沙箱，截获运行时的报错信息，并将其转化为反馈提示词，让大模型具备自主修正错误代码的能力。
2. **基于预训练模型与联合概率的复杂度感知调度策略**：为了平衡系统可靠性和 API 调用成本，提出一种融合了自然语言处理和资源调度的方案。通过收集智能体以前的执行日志建立数据集，引入 BERT 模型对用户的提问进行特征提取。在此基础上，利用多层感知机预测代码生成的成功率，并结合联合

概率公式，推导出满足目标成功率的最优并发数和重试次数，实现资源的合理分配。

3. **前后端分离的智能交互式数据查询系统实现**：基于上述理论，采用微服务架构开发一个端到端的工程系统。前端使用 Vue 框架构建交互界面，支持自然语言提问和图表渲染；后端使用 Django 框架构建业务网关，并拆分出独立的模型训练微服务。系统整合了数据库管理、模型推理和结果展示，为非技术人员提供一个门槛较低、反应较快且比较稳定的数据查询平台。

1.4 论文组织结构

本论文主要研究基于大模型的数据库智能查询技术，全文一共分为六章，每一章的具体安排如下：

- **第 1 章导论**。主要介绍研究的背景和意义，梳理国内外在 Text-to-SQL、智能代理以及资源调度方面的研究现状，并明确本文的研究目标、内容和章节结构。
- **第 2 章相关理论与技术基础**。梳理系统运行所需的理论基础。包括基于 Transformer 的模型生成机制、BERT 模型的编码原理，以及 PGVector 向量检索技术在表结构对齐中的应用。
- **第 3 章基于 Agent 的 Text-to-Python 代码生成与自修复机制研究**。重点介绍系统执行端的逻辑。包括如何将关系型数据库映射为 DataFrame 结构，以及基于沙箱的报错捕获机制，展示大模型如何通过反馈提示词进行纠错。
- **第 4 章基于 BERT 的并发预测与动态调度模型设计**。论述算法模块的构建过程。包括利用 BERT 提取复杂度特征并接入回归网络，结合概率公式推导最优并发数，并提出带早停机制的多线程调度算法。
- **第 5 章数据库高效查询原型系统实现与测试**。展示基于 Vue 与 Django 框架的系统落地细节。分析核心接口设计和连接池管理，并通过测试展示动态调度策略在降低成本和提升成功率方面的效果。
- **第 6 章结论与展望**。总结全文的研究和实践成果，评价模型在复杂业务下的局限性，并对未来的研究方向进行展望。

第 2 章 相关理论与技术基础

本系统的主要任务是去搭建一个既高效又能稳定运行的自然语言数据库查询智能体。想把这个目标落地，单单依靠大语言模型本身的语义理解和代码生成能力是不太够的；还需要把现代的自然语言处理理论、关系型数据库的底层运行机制，以及一些复杂的并发调度算法给结合进来。所以本章会对支撑系统运转的这些基础理论和核心技术做一个简单的梳理与探讨。

2.1 自然语言处理基础

自然语言处理（NLP）其实就是让人和机器能够顺畅交流的一座桥，它最根本的作用是让计算机具备去解析、理解甚至生成非结构化人类语言的能力。对于本系统这种基于智能体的查询代理来说，Text-to-Python 模块到底能不能准确抓取到用户那些多变的数据分析想法，很大程度上就得靠现代的 NLP 理论来做支撑了。

2.1.1 自然语言处理技术的历史演进

回顾自然语言处理技术的发展，其实就是一个从依靠规则驱动慢慢转向靠数据驱动的过程^[4]。早期的 NLP 系统往往高度依赖语言学家去人为编写各种词法和句法规则（比如基于语法树解析的那一套体系）；这种套用规则的方法在一些特定的垂直领域里确实比较准，但是一放到什么情况都会出现的开放场景里就很容易卡壳，遇到用户语序倒装或者省略掉几个字的时候，系统可能直接就没法处理了。

慢慢地，基于统计和机器学习的办法成了主流。人们开始把隐马尔可夫模型（HMM）、条件随机场（CRF）还有 N-gram 语言模型引进来，通过对大量的语料做词频统计以及概率分布建模，算是把规则模型泛化能力弱的问题给缓解了一些；不过统计模型依旧会被特征工程的瓶颈给限制住。到了最近这几年，随着计算机算力的大幅提升，基于神经网络的深度学习技术算是把 NLP 的发展路线彻底改变了。从一开始用卷积神经网络（CNN）来做文本分类，到后面循环神经网络（RNN）和长短期记忆网络（LSTM）在机器翻译等领域大范围普及，模型这才真正开始有了处理长文本时序信息的能力。这个转变不仅打破了人工提取特征的局限，也为后来出现的大规模预训练模型铺平了道路。

2.1.2 词嵌入技术在自然语言处理中的应用

在深度学习的计算框架下，计算机是没法直接去认识那些离散的文本符号的，必须得把它们映射进一个连续的数值空间里，这个不可或缺的前期步骤就被叫做词嵌入（Word Embedding）^[5]。以前做自然语言处理的时候，大家多半会用独热编码（One-Hot Encoding），但这方法不仅很容易引发数据稀疏和维度灾难，最麻烦的是它直接切断了词与词之间的那些内在联系——因为随便挑两个词向量去算内积，结果都是零。

为了解决这个缺陷，Word2Vec 和 GloVe 这些分布式表示技术就被提了出来。一方面，Word2Vec 会利用浅层神经网络去做自监督训练，把词汇转化成低维度的稠密向量；另一方面，GloVe 则是通过构建全局词汇共现矩阵去捕捉更多的统计特征^[6]。到了这些高维的连续空间里，只要是语义差不多或者语法功能相似的词（比如“销售额”跟“营收”，或者“国家代码”和“CountryCode”），它们算出来的欧几里得距离或余弦距离就会变得很近。词嵌入技术打破了文字符号表面的隔离，让系统在执行数据库查询时，能顺利跨越用户的日常口语表达和底层严格表结构（Schema）之间的语义障碍，在数学层面把向量化检索给跑通了。

2.1.3 提示词工程

随着大语言模型（LLM）的普及，以往那种“先预训练再微调”的传统习惯，正在逐渐被“预训练加上提示”的新模式给替代。在这种情况下，提示词工程（Prompt Engineering）慢慢成了一项能决定智能体表现上限的关键交互技术；它其实不是随便输入几个问题那么简单，而是通过去设计上下文条件、设定模型角色以及给少量样本做引导（Few-Shot Learning）^[7]，在不用修改模型底层参数的前提下，把大模型在特定任务里的推理潜力给激发出来。

近年来，随着思维链（Chain of Thought, CoT）等高级策略的出现，模型应对复杂逻辑推理的表现又上了一个台阶^[8]。而在本课题所设计的 Agent 架构当中，提示词工程早已被深度地融合进到了沙箱执行的每一个环节里，具体来说体现在两个方面：第一点是系统会把从向量库里精准查询出来的数据库表结构，也就是表名、字段名以及具体的注释信息，全都动态地给塞进到初始提示词里面，以便于让模型能够根据设计好的数据逻辑去思考；第二点则是只要生成的 Python 脚本在沙箱环境里出现报错或运行失败，Agent 就会捕捉到标准的运行时错误堆栈，并把它转化成结构化的反馈提示词再传回到大模型去，正是靠着这种通过动态提

示词来来回回不断调整的闭环机制，才使得智能体在面对开放域查询任务的时候，容错水平和更正 Bug 的自我修复能力得到了极大程度的增强。

2.2 Transformer 与 BERT 基础

当下那些大规模的预训练语言模型之所以在写代码和逻辑推理上能有不错的表现，它们底层的架构几乎都是搭建在 Transformer 上的。而 BERT 作为这套架构里比较经典的一个分支，也给本系统中独立调度模块去预测“查询复杂度”提供了很关键的特征表示基础。

2.2.1 Transformer 的历史与架构突破

在 2017 年之前，大家处理自然语言序列转换 (Seq2Seq) 任务时，主流选择基本都是循环神经网络 (RNN) 和它的一些变体。但 RNN 有个回避不了的问题，那就是隐状态的更新必须得排队等上一步的输出；这种串行的计算方式严重影响了硬件去进行大规模并行加速的效率，而且句子一长就容易出现梯度消失，导致模型抓不住远距离的依赖关系。直到 2017 年谷歌团队抛弃了循环和卷积结构，直接提出了 Transformer 模型^[9]。这个架构完全靠注意力机制去建立全局联系，很大程度上改变了深度学习的研究思路，也开启了后续大模型参数量快速增长的阶段。

2.2.2 Transformer 的核心数学机理

Transformer 架构的核心在于它引入的缩放点积自注意力机制 (Scaled Dot-Product Attention)。在分析用户的自然语言提问或者解析数据库表结构时，模型会把输入的序列映射成三个互不干扰的权重矩阵：查询矩阵 Q 、键矩阵 K 以及值矩阵 V 。把 Q 和 K 拿去计算点积并做完归一化处理，模型就能自己去衡量当前词跟上下文其他词之间的关联度有多高，核心计算公式如下：

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (2-1)$$

在这个公式里加了一个缩放因子 $\sqrt{d_k}$ (d_k 代表键向量的维度)，主要是为了防止点积算出来的结果过大，从而让 Softmax 函数掉进梯度极小的饱和区。

为了让模型可以在不同的特征空间里去捕捉更丰富的语义信息（比如同时关注代码语法和业务逻辑），Transformer 又加了一个多头注意力机制 (Multi-Head Attention)；它的原理是把高维向量投影到多个低维空间里去并行计算自注意力，

最后再把各个“头”的结果给拼接起来做个线性映射：

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (2-2)$$

其中每一个独立的注意力头 $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$ 。

另外，由于注意力机制本身是识别不了文本顺序的，但 SQL 语句或者 Python 代码对词的先后顺序又特别敏感（比如 ‘A JOIN B’ 跟 ‘B JOIN A’ 完全不一样），所以 Transformer 就用正弦和余弦函数交替的绝对位置编码（Positional Encoding）把时序信息给加了进去：

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (2-3)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (2-4)$$

特征流在经过多头注意力的处理后，最终会通过一个带 ReLU 激活函数的两层全连接前馈神经网络（FFN），以此来增强模型做非线性映射的能力：

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2-5)$$

也正是有了这些数学运算的支撑，模型才有了相对出色的深层逻辑推导能力。

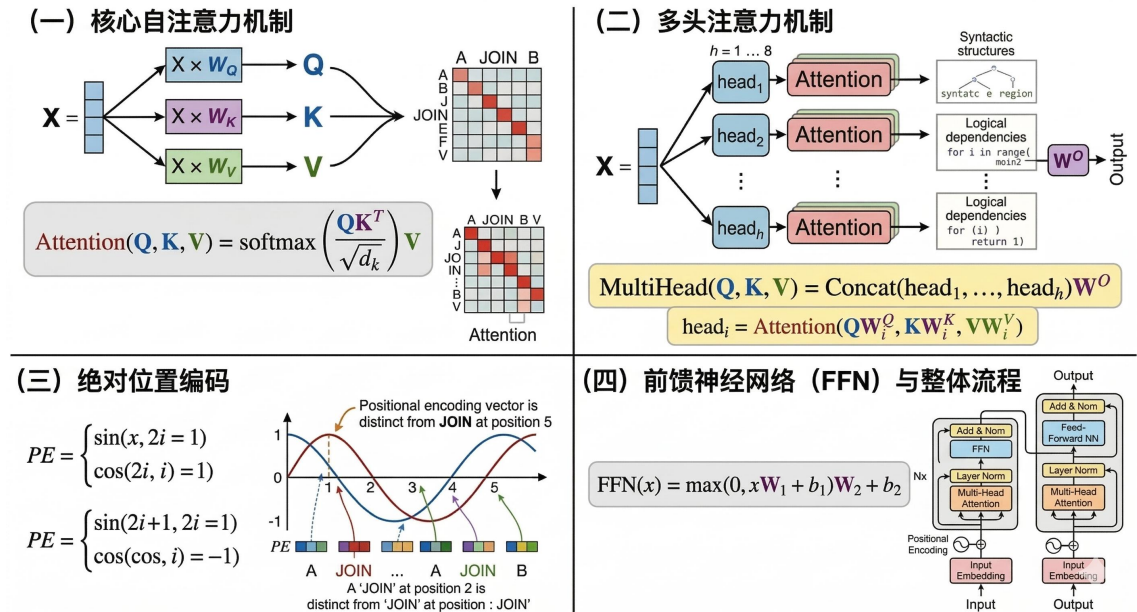


图 2.1 Transformer 核心原理与架构图

Fig. 2.1 Core Principles and Architecture of Transformer

2.2.3 从 Transformer 到 BERT 的演进

最开始的 Transformer 是由编码器 (Encoder) 和解码器 (Decoder) 这两个大模块组成的。在后来的技术发展中,学术界慢慢走出了两条不同的路线:一条是以 GPT 为代表的生成式模型,它们去掉了编码器,全靠掩码自注意力机制来实现连续自回归文本生成;另一条则是 BERT (Bidirectional Encoder Representations from Transformers),它保留了多层编码器并把它们堆叠了起来^[10]。

对于本文研究的并发调度任务来说,并发预测本质上是个回归判别问题,系统需要一眼看完用户发来的完整查询指令和被提取出的数据库 Schema 摘要,而不是像 GPT 那样逐词生成。BERT 这种双向融合信息的架构,刚好让它在处理“复杂度评估”和“特征回归”这种任务时显得非常有优势。

2.2.4 BERT 的输入表示与特征浓缩架构

为了让模型拿到各种复杂的句子组合都能处理好, BERT 在输入层设计了比较严谨的特征融合机制;不管输入什么文本序列,它底层的嵌入表示 (Embedding) 实际上是由词元嵌入、句子片段嵌入以及位置嵌入这三者在向量空间里逐个元素相加构成的一个综合张量:

$$E = E_{\text{token}} + E_{\text{segment}} + E_{\text{position}} \quad (2-6)$$

获取到输入张量后,模型会通过掩码语言模型 (MLM) 和下一句预测 (NSP) 这两大无监督任务去自己训练参数。在工程实现上,系统会在输入序列的最前面强行加一个特殊的分类标记 [CLS];虽然这个标记本身没什么实际字义,但等它在一层层双向 Transformer 编码器里经过密集交叉注意力计算后,它对应的隐藏层输出向量就等于是把整段输入序列的全局依赖关系和逻辑复杂度给浓缩了进去。如果假设输入字符映射后的特征序列是 $E_1, E_2, \dots, E_n$, 那它那个稠密向量表示 h_{cls} 算起来就是:

$$h_{cls} = \text{BERT}_{\text{Encoder}}([\text{CLS}], E_1, E_2, \dots, E_n) \quad (2-7)$$

在本项目独立拆分出来的 Training 微服务里,系统用来量化查询难度、推算联合概率并分配并发线程数的依据,靠的也就是这个包含了丰富信息的 768 维特征向量 h_{cls} ,它会被直接送进下游的 MLP 回归网络里作为计算的核心基石。

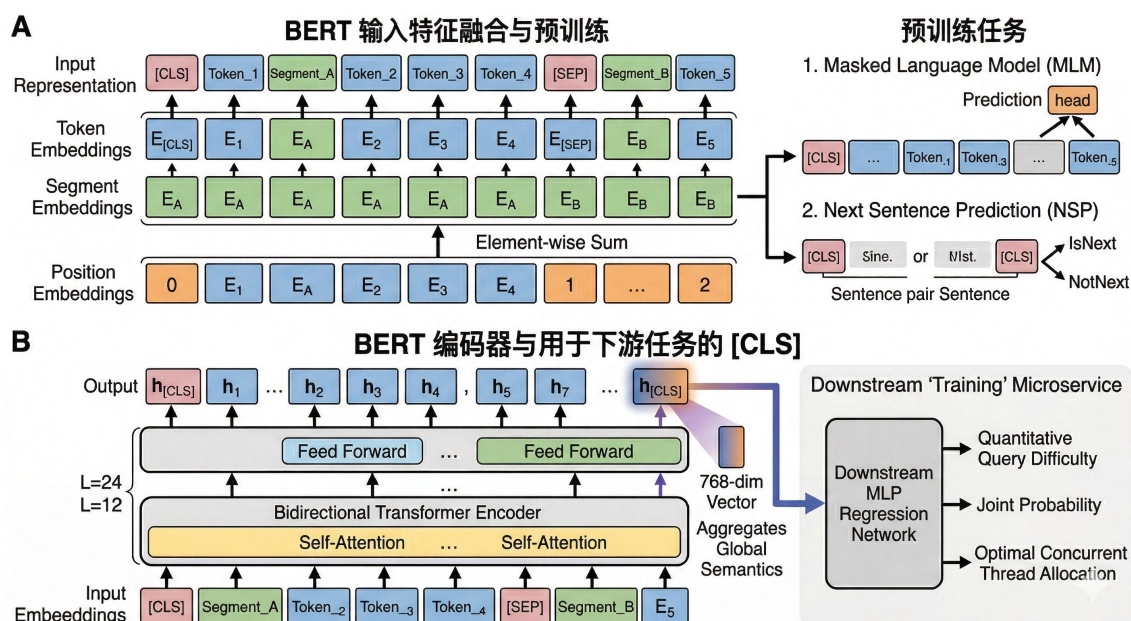


图 2.2 BERT 核心原理与架构图
Fig. 2.2 Core Principles and Architecture of BERT

2.3 数据库理论基础

如果要搭建一个能处理复杂业务的智能查询代理，不仅要考虑大量结构化数据怎么进行事务性存储，还得解决自然语言和底层模式库在跨模态语义上怎么匹配的问题。因此本系统在底层的数据架构设计上，把传统关系型数据库和现代向量数据库的特性给结合到了一起。

2.3.1 关系数据库基本概念

自从 1970 年提出关系模型以后，关系型数据库（RDBMS）就凭借其严密的数学基础，变成了企业级数据管理的核心^[11]。它的核心思路就是把现实业务里的实体和关联关系都抽象成二维表格，然后再通过主键和外键去限制它们之间的参照完整性。同时，关系模型会强制推行 ACID（原子性、一致性、隔离性、持久性）标准，这就很好地保障了高并发读写场景下的数据安全和恢复能力。不过这种严格的模式约束虽然保证了稳定，却也给不懂技术的用户拉高了使用门槛；为了降低门槛，SQL 作为一种声明式语言被推行开来，主要用来让底层的查询优化器去执行跨表联接或者聚合之类的复杂运算。

2.3.2 MySQL 架构设计

MySQL 凭借着插件式的存储引擎架构，在开源关系型数据库领域占有很重要的位置。它的逻辑架构自上而下一般分为三层：网络连接层、核心服务层以及存储引擎层，这种解耦的设计让系统在面对复杂查询和高并发读写时有很强的稳定性和扩展性。

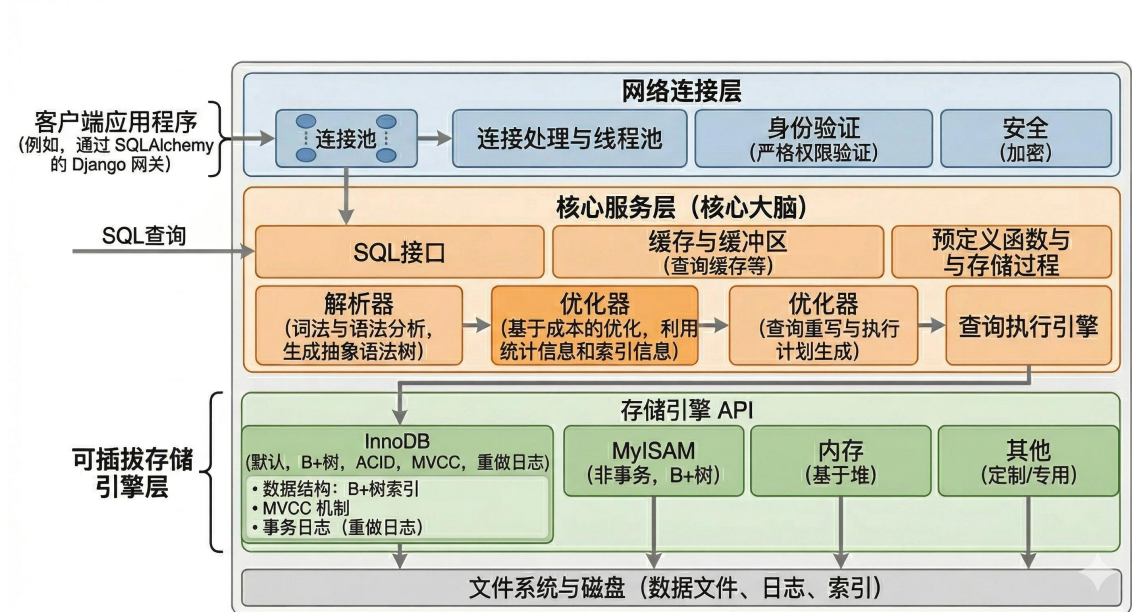


图 2.3 MySQL 逻辑架构与查询流程图

Fig. 2.3 MySQL Logical Architecture and Query Flow

最上方的网络连接层主要负责跟外部客户端通讯，并提供连接池管理、身份验证和安全加密等服务；当后端系统通过框架发起连接请求时，这一层会分配线程来维持会话，并严格核对用户权限，算是筑起了底层数据的第一道防线。

中间的核心服务层是 MySQL 解析并优化查询指令的大脑；这里面包含了 SQL 接口、解析器、查询优化器以及各类内置函数。SQL 语句进来后会先被解析成抽象语法树，接着优化器会参考表结构和索引情况去重写查询逻辑，最后生成一个最优的执行计划；虽然本课题的智能体多数时候是在沙箱里操作 DataFrame，但前期用 Pandas 从 MySQL 抽取海量业务数据时，依然要依靠这个服务层去进行高效调度。

最下面的存储引擎层则负责数据在物理层面的存储和提取；MySQL 允许不同的数据表配置不一样的存储引擎，目前默认用的 InnoDB 引擎底层是基于 B+ 树的结构，它不仅支持高效的范围查找，还能通过多版本并发控制 (MVCC) 和重做日志把 ACID 事务特性给支撑起来。有了这种稳固的存储架构，上面智能体去进

行大规模数据吞吐时就有了很扎实的物理基础。

2.3.3 向量数据库基本概念

传统关系型数据库的那套 B+ 树或哈希索引，一般只能用来做精确匹配和简单的一维检索，一旦遇上大语言模型生成的高维稠密向量，这种老式的索引机制就会因为“维度灾难”而失效。向量数据库刚好就是为了解决高维空间内非结构化数据怎么算相似度而出现的；它不去死磕字符符号的完全一致，而是依靠近似最近邻（ANN）算法，在海量的高维数组里快速找出距离最近的目标集。在距离度量上，系统一般会用欧氏距离、内积或余弦距离来评估两个特征向量在语义上有多重合。引入了这个特性后，跨模态的特征检索和自然语言的模糊匹配在工程上才变得可行。

2.4 智能体架构理论与设计

以前常见的自然语言处理模型，大多都是作为静态的输入输出函数来运行的，缺乏感知外部环境和动态交互的能力。而基于大语言模型的智能体则通过引入工具调用和记忆机制，从单纯的“文本生成器”变成了能够自主决策的代理。本文的研究结合了关系型数据库的底层特点，设计出了一套具备环境感知、沙箱隔离以及自我评估能力的闭环智能体架构。

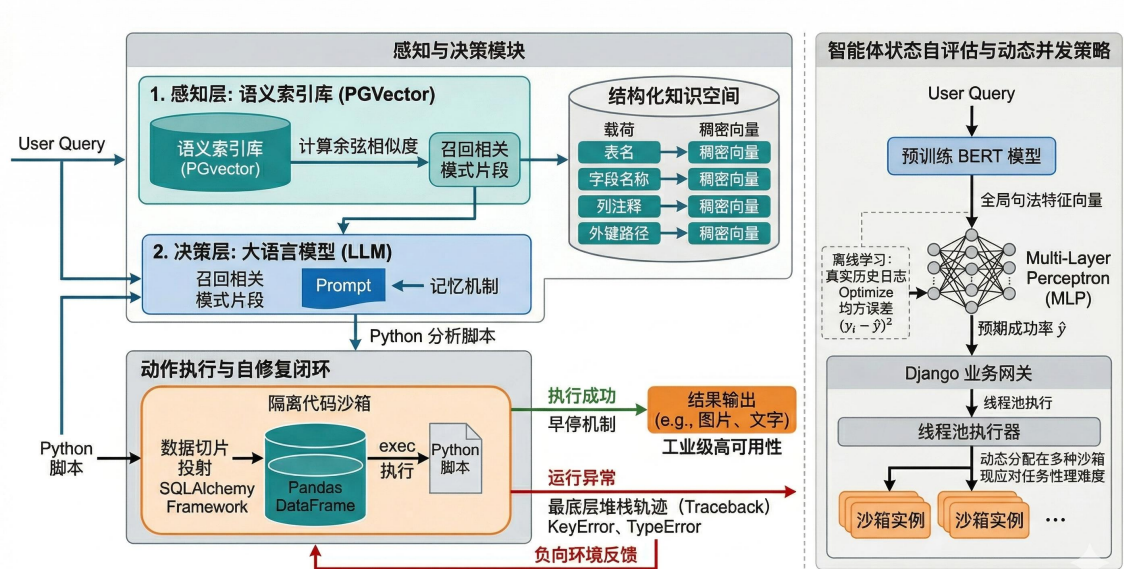


图 2.4 智能体总体交互架构图

Fig. 2.4 Overall Interaction Architecture of LLM-Based Agent for Database Analysis

2.4.1 环境感知与结构知识索引

智能体要去执行复杂任务，第一步就是要对所处的环境有个准确的认识，这在学术上一般叫做解决“符号接地问题”；面对包含着大量数据表和复杂关联的业务数据库，如果把所有的模式信息毫无节制地全都丢进大模型的上下文里，不仅会导致 Token 溢出，还会让模型的注意力衰减，最后直接把前面的关键指令给忽略了。

基于这个考虑，本系统在智能体的“感知层”里加入了一个基于 PGVector 的结构化语义检索模块；跟那种不断累积的长期记忆系统不太一样，本系统构建的是一个专门面向数据库结构知识的索引库。系统在运行的时候，会通过哈希去重机制把动态加载进来的表名、字段名、注释以及外键路径等结构全给编码成稠密向量，然后再存到数据库的特定集合里。

等处理用户抛出来的数据库分析需求时，智能体就会利用余弦相似度算法，在知识空间里精确地去量一下语义距离：

$$\text{Similarity}(q, k) = \frac{q \cdot k}{\|q\| \|k\|} \quad (2-8)$$

在这个基础上，系统底层的建表脚本虽然已经通过 ivfflat 等机制给近似最近邻检索提供了支持^[12]，但在普通的表规模下，智能体通常还是依靠通用的相似度检索接口去直接召回相关度最高的信息。

通过这种检索增强的机制，智能体可以在很短的时间内实现注意力聚焦，把环境里的无关噪声表给精准过滤掉；提取出来的模式摘要会被动态放进提示词里，这大幅提升了大模型在生成代码时选择字段和关联路径的准确率，给后续的逻辑推理划定了一个明确的边界。

2.4.2 基于反馈的动作执行与自修复闭环

确定好环境上下文后，智能体就进入了“规划与执行”（ReAct）阶段^[13]。传统的 Text-to-SQL 代理往往倾向于直接生成底层数据库语言去执行，这种没有中间防线的做法其实风险很高，很容易引发数据污染或者提权攻击；为了解决这个问题，本系统创新性地设计了 Text-to-Python 的中间层隔离执行模式。

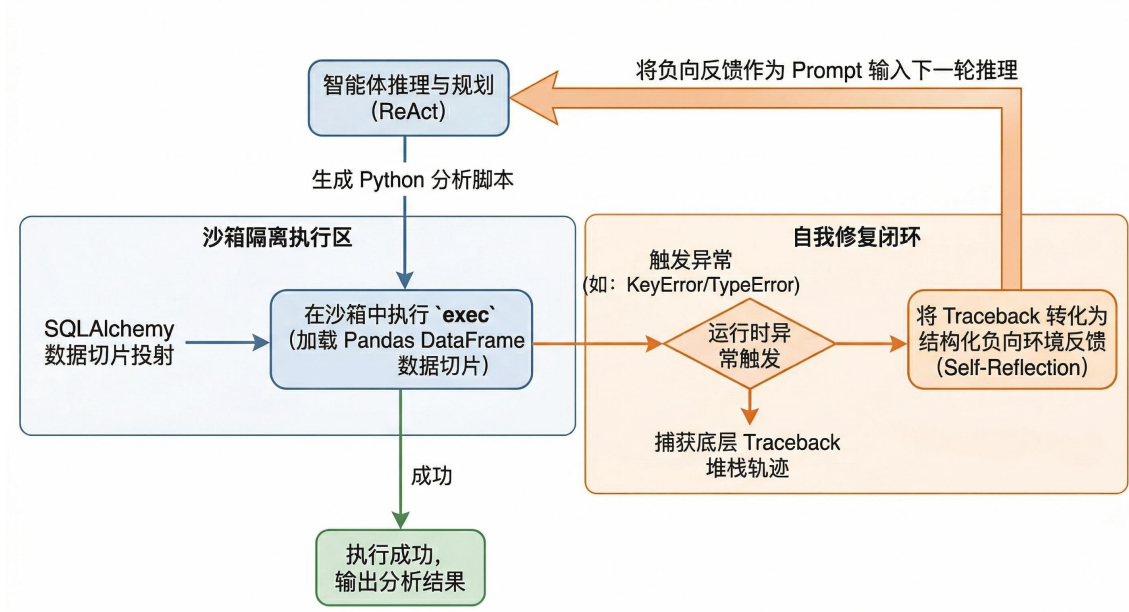


图 2.5 智能体沙箱执行与自修复闭环状态图

Fig. 2.5 State Diagram of Agent Sandbox Execution and Self-Correction Closed-Loop

系统里的智能体并不会直接接触物理数据底座，而是利用 SQLAlchemy 框架把数据转化成内存里的 Pandas DataFrame 对象；然后它生成的 Python 分析脚本会被限制在有资源隔离的代码沙箱里通过 exec 算子去执行。更重要的是，这个智能体具备了从失败中总结反思的能力 (Reflexion)；只要生成的逻辑在运行中碰到了 KeyError 或者 TypeError 之类的异常，底层的执行器就会立刻截获最原始的堆栈报错信息。这些报错信息会被智能体整理成结构化的负向环境反馈，接着无缝传回给下一轮推理的提示词里；这种“生成-执行-失败-重构”的动态闭环，让模型拥有了类似人类开发者那样在线调试和修复问题的能力。

2.4.3 智能体状态自评估与动态并发策略

大语言模型这种基于概率的自回归生成特性，决定了它在推导特别复杂的嵌套业务逻辑时，成功率往往是不太稳定的。为了解决单一智能体不够可靠的瓶颈，系统引入了自评估模型以及一套冗余并发调度机制。

和以往靠人工经验去固定线程数的粗放策略不同，本研究把评估自然语言任务难度的问题，转化成了一个特征回归任务。具体来讲，系统使用了预训练好的 BERT 模型作为特征提取器，把输出的全局句法特征向量接入到多层感知机 (MLP) 网络里，通过前向传播去动态预测当前任务在单智能体环境下的成功率^[14]：

$$\hat{y} = W^T \sigma(V^T h_{cls} + b_1) + b_2 \quad (2-9)$$

在离线学习的时候，这个评估模型会以真实的历史执行日志作为参考，采用均方误差去计算预测值和真实成功率之间的差距，顺着梯度下降不断去优化权重参数：

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2 \quad (2-10)$$

有了这套自评估网络，后端的网关就能根据任务难度，动态分配沙箱实例的数量去进行多智能体的并发试错；再配合早停机制，只要有一个智能体先解出了答案，系统就会迅速关掉其他的冗余线程，这样不仅保障了系统的高可用性，也把底层算力分配得比较精准。

2.5 本章小结

这一章系统地梳理了构建大语言模型数据库查询代理所需要的底层理论和核心技术。从自然语言处理的技术演进和词嵌入技术切入，本章相对通俗地剖析了基于 Transformer 架构的自注意力机制以及 BERT 模型的语义表征原理，这为系统后续去量化查询意图和评估任务复杂度打好了算法底子。在这个基础上，本章又探讨了传统关系型数据库的架构设计，以及现代向量数据库的高维检索逻辑，理清了如何通过“关系加向量”的双引擎底座去帮大模型解决符号接地问题。最后，本章把前面提到的理论做了个工程化的整合，详细论述了智能体在环境感知、沙箱反思闭环以及动态并发评估这几个关键环节上的设计思路；这些技术底座的结合不仅在理论上验证了 Text-to-Python 代理自修复方案的可行性，也为后续并发预测模型的训练和系统原型落地提供了指导依据。

第 3 章 基于 Agent 的 Text-to-Python 代码生成与自修复机制研究

以前在做自然语言查询数据库的研究时，大家基本上都会去采用 Text-to-SQL 这个模式^[1]；不过直接让大语言模型去写并执行 SQL 语句其实会有比较高的 SQL 注入风险，而且 SQL 本身在图灵完备性上有些欠缺，碰到那种复杂的数据可视化或者时序聚合的需求时就很难直接去处理了。考虑到这些问题，本系统更换了思路，引入了一套 Text-to-Python 的中间层代理（Agent）架构，相当于是把大语言模型的角色从单纯的代码生成者提升至数据分析师的角色。所以在这一章里，主要就会详细梳理一下这个 Agent 架构是怎么设计的，还会探讨基于 PGVector 怎么去做动态的上下文感知，以及遇到代码报错时怎么利用异常信息去完成闭环修复。

3.1 Text-to-Python 代理决策框架设计

如今的大型语言模型因为吃透了海量的代码库数据，其实已经具备了相当不错的程序逻辑推理和编写 Python 代码的能力^[15]。本系统在搭建 Agent 框架的时候，最核心的想法就是把数据库里那些错综复杂的关系表安全地拆解开，然后映射成系统内存里的 Pandas DataFrame 对象^[16]。

有了这层架构之后，大模型就不需要再直接去跟底层的数据库引擎做那种风险很高的 SQL 交互了；它只需要去写一些调用 Pandas 库的 Python 脚本，就能把数据的清洗、过滤还有复杂的聚合计算给做完。这种在沙箱里隔离开来执行的中间层模式，一方面拉高了系统防御恶意指令的安全线，另一方面也给后面去做自动化的代码诊断和抓取报错异常留足了操作空间。

3.2 动态 Schema 感知与 DataFrame 映射

为了让大模型在消耗较少 Token 的情况下准确理解数据库的库表设计与数据，系统必须要在生成 Python 脚本之前，就把准确的数据模式作为限制条件加进提示词里。

3.2.1 MySQL 物理模型到 DataFrame 的内存级映射

在大部分的企业级应用里，MySQL 基本都是用来承担高并发和数据持久化存储任务的，它底层的数据其实就是靠 B+ 树以及格式较为死板的行存二维表来存放的；只是这种偏向于磁盘读写和严格事务管理的物理结构，天生就不太适合直

接拿给大模型去当动态分析的沙箱用。为了弥补底层数据库和 Python 灵活分析环境之间的差异，系统里的数据访问模块借助了 SQLAlchemy 这个框架，把 MySQL 里的物理表高效地转成了 Pandas DataFrame 这种内存对象。

具体操作起来的话，系统会利用数据库嗅探器去动态扫一遍 MySQL 里所有正在用的表结构，并且把字段名精准地提取出来，防止后面出现那种预料之外的数据类型解析报错；接着系统会动态生成一个比较安全的数据抽取查询指令，利用 `pd.read_sql` 把核心的业务数据一次性全给加载到应用服务器的物理内存里。把数据搬到内存这一步，其实就是把原本那种在数据库里很麻烦的多表联查操作，降维成了很灵活的 DataFrame 内存拼接和矩阵计算；这从物理边界上断绝了大模型乱写恶意 SQL 去破坏底层引擎的可能，保障了核心数据的安全。

```
1 def _load_tables_data(inspector=None) -> Dict[str, pd.DataFrame]:
2     """Read all database tables into pandas DataFrames.
3
4     Args:
5         None.
6
7     Returns:
8         dict[str, pandas.DataFrame]: Mapping from table name to
9         full table DataFrame.
10
11     Raises:
12         RuntimeError: If table listing or data querying fails.
13     """
14     try:
15         inspector = inspector or inspect(engine)
16         loaded_tables: Dict[str, pd.DataFrame] = {}
17         with engine.connect() as connection:
18             for table_name in inspector.get_table_names():
19                 columns = [str(column.get("name")) for column in
20                             inspector.get_columns(table_name)]
21                 query = _build_select_all_query(table_name, columns)
22                 loaded_tables[table_name] = pd.read_sql(query,
23                                                         connection)
24         logger.info("Loaded %d tables from database", len(
```

```

loaded_tables))
22     return loaded_tables
23 except Exception as exc:
24     raise RuntimeError(f"Failed to load table data from
        database: {exc}") from exc

```

Listing 1 数据库表数据到 Pandas DataFrame 的动态加载

3.2.2 基于 PGVector 的结构知识语义召回

如果遇到那种业务表特别多的数据库，要是毫无节制地把所有的表结构信息全都塞给大模型，很容易就会导致上下文超载并且让大模型的逻辑跟着变乱。为了解决模型弄不清复杂表结构和外键关联的问题，本系统就在 Agent 的决策流程里加入了检索增强生成（RAG）机制^[17]。

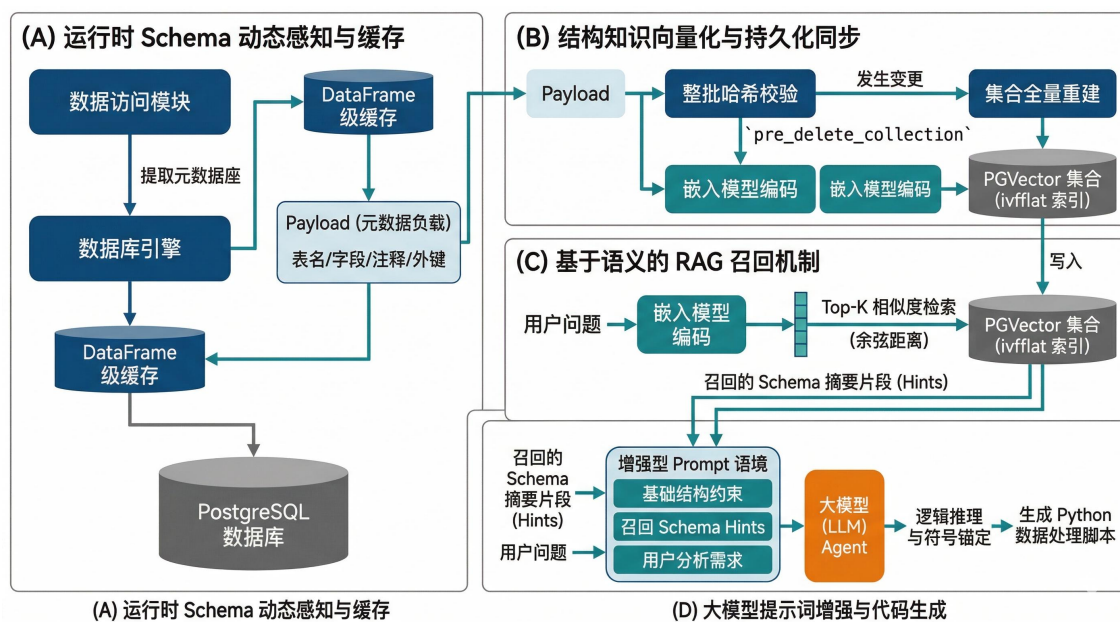


图 3.1 基于 PGVector 的 RAG 检索增强架构图

Fig. 3.1 PGVector-based RAG Retrieval Enhancement Architecture

在代码落地的阶段，系统会把运行的时候动态加载进来的表名、字段注释还有外键路径这些信息，先给转换成那种高维度的稠密向量，等过了哈希校验之后，就会同步存到 PostgreSQL 的 PGVector 扩展插件里去；等到系统接收到了比较复杂的数据分析请求，Agent 就会直接去调用向量库的 Top-K 相似度接口，计算余弦距离，凭借底层近似最近邻索引的加速特性，用极短的时间把相关度最高的表结构片段给找出来，然后追加进主提示词的上下文里面。有了这条很轻量级的语

义补充链路，能在很大程度上避免大模型去“凭空捏造”一些不存在的字段，从而为生成 Python 脚本的推导过程定下了一个比较靠谱的逻辑基调。

3.3 Text-to-Python 代码生成架构设计

大语言模型在面对逻辑严密的数据处理任务时，它最后能写出多好的代码，很大程度上取决于目标语言是怎么表达的以及给它的上下文限制是否明确。这一节就重点梳理一下系统把自然语言转成 Python 代码的核心逻辑。

3.3.1 任务降维：从声明式 SQL 到命令式 Pandas 的范式转换

以前用那种传统的 Text-to-SQL 技术时，都是硬性要求模型得一次性生成一整句完整的查询代码。可一旦碰上那种需要写好几层子查询、用到复杂的窗口函数还得加条件做聚合的报表需求时，SQL 语言本身那种太死板的语法结构，往往会让大模型在整理逻辑的时候产生错误，很容易就把表关联写错了，或者把该聚合的维度给漏掉了。

为了从这个困境里跳出来，本系统换了个新方向，把要生成的代码目标从 SQL 切换成了能去操作 Pandas 库的 Python 脚本^[18]。因为 Python 是图灵完备的编程语言，它可以把那些极其复杂的数据处理工作给拆开，变成一行一行按顺序去执行的变量赋值或者是数据过滤操作；这种把大问题拆成小步骤的做法，非常契合大模型那种喜欢顺着思维链往下推导的特性。在系统真正跑起来的时候，底层数据库里的表其实早就被映射成 `data_dict` 这个字典里面的 `DataFrame` 对象了，大模型需要操心的仅仅是怎么去调几个类似 `df.groupby().sum()` 这种接口把字典里的数据切块或者重新拼起来；这么做不仅把模型推理的难度给降下来了，还顺带拥有了后续去做复杂数学运算和画图表的能力。

3.3.2 提示词模板工程与动态上下文组装

在系统进入自动化代码生成的核心执行阶段时，提示词的设计早就超越了对用户自然语言指令的简单映射，而是演进为了一套具备严密语义逻辑与格式规范的系统级通信协议；为了确保大语言模型在受限的 Token 窗口内，把最为关键的领域知识与数据特征给精准地捕获到，本研究于系统后端专门构建了一套提示词动态上下文组装引擎。

如代码清单 2 所示，该引擎底层的 `get_final_prompt` 函数通过把复杂的提示词结构给解构成了多个功能明确的信息模块，并依据运行时的数据状态来进行

动态拼装。具体而言：第一点是利用 `pre_prompt` 把针对 `pandas` 数据处理的任务指令给预先设定好；第二点是通过调用 `_slice_dfs` 方法把物理数据库中真实的 `DataFrame` 给切片提取出少量的样本数据 (`data_slice`)，从而把模型推断数据类型时可能产生的逻辑偏移给有效规避掉；第三个方面则是基于动态传入的 `data` 列表，把隐含的外键关联约束 (`data[1]`) 与表字段注释信息 (`data[2]`) 给一并注入到上下文里，以极大地增强大模型的符号接地能力；紧接着，系统会专门调用 `_append_semantic_context` 接口，把由 `PGVector` 向量引擎召回的高相关度表结构语义提示给动态地追加进去；最后则是利用 `end_prompt` 把输出格式给严格限制在单一的 `Markdown` 代码块内，并强制要求模型遵循尽早过滤、避免笛卡尔积等查询优化原则，从工程层面很好地保障了生成代码的可执行性与运行效率。

```
1 def get_final_prompt(data: List[Any], question: str) -> str:
2     """Assemble the final prompt sent to the model for Python code
3         generation.
4
5     Args:
6         data (list[Any]): Input payload where 'data[0]' is a
7             DataFrame dict,
8             'data[1]' optionally contains key constraints, and '
9             data[2]' optionally contains comments.
10        question (str): Task instruction text appended to the
11            prompt template.
12
13    Returns:
14        str: Fully composed prompt including sampled data, optional
15            key constraints,
16            optional comments, optional semantic vector hints, and
17            output format constraints.
18    """
19    data_rows = int(config_data["ai"]["data_rows"])
20    data_dict: Dict[str, pd.DataFrame] = {}
21    if data and isinstance(data[0], dict):
22        data_dict = data[0]
23
24    data_slice = _slice_dfs(data_dict, max(1, data_rows))
25    pre_prompt = (
```

```

20         "Write a Python function called process_data that takes
           only a pandas "
21         "dataframe dictionary called dataframes_dict as input and
           performs the following operations:\n"
22     )
23
24     data_prompt = (
25         "Here is the dataframe dictionary sample "
26         "(it is only structural sample data, not full production
           data):\n"
27     )
28
29     key_prompt = (
30         "The tables may be connected through columns with different
           names, "
31         "and those relations are defined as key constraints.\n"
32         "Format: {'table_name': {'table_name.column ': '
           referred_table.referred_column'}}\n"
33     )
34
35     comment_prompt = "Here are table and column comments for
           semantic reference:\n"
36
37     end_prompt = (
38         "Return code in a single markdown code block only. "
39         "Do not include explanations, shell commands, or comments.
           "
40         "Do not call the function. Do not print anything. "
41         "Import required modules inside the function. "
42         "Do not mock data. "
43         "Apply query optimization principles: filter rows as early
           as possible before merge/groupby/sort, "
44         "select only required columns before joins, avoid Cartesian
           products, and avoid unnecessary full-table sorting."
45     )
46

```

```

47     all_prompt = pre_prompt + question + "\n" + data_prompt + str(
        data_slice)
48     if len(data) > 1 and data[1]:
49         all_prompt += "\n" + key_prompt + str(data[1])
50     if len(data) > 2 and data[2]:
51         all_prompt += "\n" + comment_prompt + str(data[2])
52
53     all_prompt = _append_semantic_context(all_prompt, question)
54     all_prompt += "\n" + end_prompt
55     return all_prompt

```

Listing 2 基于项目源码的动态提示词组装引擎实现

正是靠着这种分层次、模块化的动态组装机制，系统才成功地把原本模糊且不可控的自然语言意图，给强行收束到了一条具备高度确定性的工程化决策路径之上。

3.3.3 基于安全沙箱的隔离执行机制

因为系统需要去直接运行大模型动态生成出来的脚本，可大模型有时候生成的代码并不完全靠谱，这就不可避免地会面临远程代码执行带来的安全威胁^[19]。如果不去好好加以限制的话，带有恶意的代码就可能去调用系统的内置模块，修改文件内容或者泄露环境变量。

考虑到这一点，系统就在 Agent 的执行引擎层搭了一个有着严格隔离机制的 Python 内存沙箱，它底层的接管逻辑就如代码清单 3 所示的那样：

```

1 def _execute_generated_code(ans_code: str, data_dict: Dict[str, pd.
    DataFrame]) -> Any:
2     """Execute generated Python code in an isolated subprocess
        sandbox.
3
4     Args:
5         ans_code (str): Generated Python function code.
6         data_dict (dict[str, pandas.DataFrame]): DataFrame
            dictionary passed to ‘process_data’.
7
8     Returns:
9         Any: Return value from ‘process_data’.

```

```

10
11     Raises:
12         ValueError: If ‘‘process_data‘‘ is missing or not callable.
13         Exception: Propagates any exception raised by generated
14             code.
15     """
16     import os
17     import pickle
18     import signal
19     import subprocess
20     import sys
21     import tempfile
22     import textwrap
23
24     payload = pickle.dumps((ans_code, data_dict), protocol=pickle.
25         HIGHEST_PROTOCOL)
26
27     error_fd, error_path = tempfile.mkstemp(prefix="sandbox_error_"
28         , suffix=".bin")
29     os.close(error_fd)
30
31     sandbox_code = textwrap.dedent(
32         """
33         import pickle
34         import os
35         import resource
36         import sys
37         import traceback
38
39         _ALLOWED_MODULES = {
40             "collections",
41             "datetime",
42             "functools",
43             "io",
44             "itertools",
45             "math",
46             "matplotlib",

```

```

43         "matplotlib.pyplot",
44         "numpy",
45         "os",
46         "pandas",
47         "pathlib",
48         "PIL",
49         "re",
50         "seaborn",
51         "statistics",
52     }
53
54     def __restricted_import(name, globals=None, locals=None,
55                             fromlist=(), level=0):
56         root_name = name.split(".", 1)[0]
57         if root_name not in _ALLOWED_MODULES:
58             raise ImportError(f"Import of {name!r} is blocked
59                               in sandboxed execution.")
60         return __import__(name, globals, locals, fromlist,
61                             level)
62
63     _SAFE_BUILTINS = {
64         "__import__": __restricted_import,
65         "__build_class__": __build_class__,
66         "abs": abs,
67         "all": all,
68         "any": any,
69         "bool": bool,
70         "dict": dict,
71         "enumerate": enumerate,
72         "Exception": Exception,
73         "BaseException": BaseException,
74         "float": float,
75         "getattr": getattr,
76         "hasattr": hasattr,
77         "int": int,
78         "isinstance": isinstance,

```



```

76         "issubclass": issubclass ,
77         "KeyError": KeyError ,
78         "len": len ,
79         "list": list ,
80         "map": map ,
81         "max": max ,
82         "min": min ,
83         "object": object ,
84         "print": print ,
85         "range": range ,
86         "reversed": reversed ,
87         "set": set ,
88         "slice": slice ,
89         "sorted": sorted ,
90         "str": str ,
91         "sum": sum ,
92         "tuple": tuple ,
93         "TypeError": TypeError ,
94         "ValueError": ValueError ,
95         "zip": zip ,
96         "__name__": "builtins" ,
97     }
98
99     def _limit_resources() -> None:
100         try:
101             resource.setrlimit(resource.RLIMIT_CPU, (10, 10))
102         except Exception:
103             pass
104         try:
105             memory_limit = 4 * 1024 * 1024 * 1024
106             resource.setrlimit(resource.RLIMIT_AS, (
107                 memory_limit, memory_limit))
108         except Exception:
109             pass
110
111     def main() -> None:

```

```

111     __limit_resources()
112     ans_code, data_dict = pickle.loads(sys.stdin.buffer.
113         read())
114
115
116     import matplotlib
117
118     matplotlib.use("Agg")
119     import matplotlib.pyplot as plt
120     import pandas as pd
121
122     execution_namespace = {
123         "__builtins__": _SAFE_BUILTINS,
124         "pd": pd,
125         "plt": plt,
126     }
127
128     try:
129         exec(ans_code, execution_namespace,
130             execution_namespace)
131         process_data = execution_namespace.get("
132             process_data")
133         if not callable(process_data):
134             raise ValueError(
135                 "Generated code must define a callable
136                 function named process_data."
137             )
138
139         result = process_data(data_dict)
140         sys.stdout.buffer.write(
141             pickle.dumps(("ok", result), protocol=pickle.
142                 HIGHEST_PROTOCOL)
143         )
144     finally:
145         plt.close("all")
146
147 if __name__ == "__main__":

```

```

142         try:
143             main()
144         except Exception as exc:
145             payload = pickle.dumps(
146                 (
147                     "error",
148                     type(exc).__name__,
149                     str(exc),
150                     traceback.format_exc() ,
151                 ),
152                 protocol=pickle.HIGHEST_PROTOCOL,
153             )
154             error_path = os.environ.get("SANDBOX_ERROR_PATH")
155             if error_path:
156                 try:
157                     with open(error_path, "wb") as error_file:
158                         error_file.write(payload)
159                 except Exception:
160                     pass
161             os.write(1, payload)
162             os._exit(1)
163         """
164     ).strip()
165
166     process = subprocess.Popen(
167         [sys.executable, "-I", "-c", sandbox_code],
168         stdin=subprocess.PIPE,
169         stdout=subprocess.PIPE,
170         stderr=subprocess.PIPE,
171         env={**os.environ, "SANDBOX_ERROR_PATH": error_path},
172         start_new_session=True,
173     )
174
175     try:
176         stdout_bytes, stderr_bytes = process.communicate(input=
            payload, timeout=30)

```

```

177 except subprocess.TimeoutExpired as timeout_error:
178     try:
179         os.killpg(process.pid, signal.SIGKILL)
180     except Exception:
181         process.kill()
182     stdout_bytes, stderr_bytes = process.communicate()
183     raise TimeoutError("Sandboxed code execution timed out
        after 30 seconds.") from timeout_error
184
185 if stdout_bytes:
186     try:
187         status, *details = pickle.loads(stdout_bytes)
188     except Exception as exc:
189         stderr_text = stderr_bytes.decode("utf-8", errors="
        replace") if stderr_bytes else ""
190         structured_error = None
191         if os.path.exists(error_path):
192             try:
193                 with open(error_path, "rb") as error_file:
194                     error_bytes = error_file.read()
195                 if error_bytes:
196                     status, *details = pickle.loads(error_bytes
        )
197                     if status == "error" and len(details) >= 3:
198                         error_type, error_message,
199                             traceback_text = details[0], details
200                             [1], details[2]
201                         structured_error = RuntimeError(
202                             f"Sandboxed execution failed with {
203                                 error_type}: {error_message}\n{
204                                     traceback_text}"
205                         )
206             except (OSError, pickle.PickleError, EOFError,
207                     ValueError):
208                 pass
209         if structured_error is not None:

```

```

205         raise structured_error from exc
206     raise RuntimeError(
207         "Sandboxed execution produced an unreadable
208         response. "
209         f"stderr: {stderr_text}"
210     ) from exc
211
212     if status == "ok":
213         return details[0]
214
215     if status == "error" and len(details) >= 3:
216         error_type, error_message, traceback_text = details[0],
217             details[1], details[2]
218         raise RuntimeError(
219             f"Sandboxed execution failed with {error_type}: {
220                 error_message}\n{traceback_text}"
221         )
222
223     stderr_text = stderr_bytes.decode("utf-8", errors="replace") if
224         stderr_bytes else ""
225
226     if process.returncode != 0:
227         structured_error = None
228         if os.path.exists(error_path):
229             try:
230                 with open(error_path, "rb") as error_file:
231                     error_bytes = error_file.read()
232                 if error_bytes:
233                     status, *details = pickle.loads(error_bytes)
234                     if status == "error" and len(details) >= 3:
235                         error_type, error_message, traceback_text =
236                             details[0], details[1], details[2]
237                         structured_error = RuntimeError(
238                             f"Sandboxed execution failed with {
239                                 error_type}: {error_message}\n{
240                                     traceback_text}"
241                         )

```

```

234         except (OSError, pickle.PickleError, EOFError,
235                 ValueError):
236             pass
237         if structured_error is not None:
238             raise structured_error
239         raise RuntimeError(
240             "Sandboxed execution failed without returning a
241             structured error. "
242             f"stderr: {stderr_text}"
243         )
244
245     raise RuntimeError("Sandboxed execution finished without
246                       returning a result.")

```

Listing 3 基于受限命名空间的 Agent 安全代码沙箱执行机制

在这个彻底隔离开来的运行环境里，系统强行剔除了动态导入这种危险操作，沙箱里面能用的就只有当前完成任务必须要用到的模块句柄；这么一来，就算大模型遭遇了恶意的提示词注入攻击，试图去写一些具有破坏性的代码，系统也会因为找不到模块而直接报错，从而把执行过程给截断。这个机制在把大模型写代码的能力释放出来的同时，也守住了一条很稳妥的安全底线。

3.4 基于异常回输的代码自修复闭环机制

虽说大语言模型的推理能力确实令人赞不绝口，但它基于概率去逐字生成的特性，注定了它没法完全避开那些细微的语法或者拼写错误。为了应对这个情况，系统给 Agent 安排上了类似人类程序员平时找 bug 的那种“自我调试”能力^[20]。

3.4.1 Agent 自修复核心轮询逻辑

在处理核心业务的链路里，系统利用了一个加了次数限制的重试循环逻辑，把代码沙箱里弹出来的那些导致程序运行错误的报错给抓取过来，然后把这些错误当成一种负向反馈，反向塞回到提示词里让模型自己去动态纠错。

```

1 for attempt_index in range(max_retries + 1):
2     attempts_used = attempt_index
3     prompt_with_feedback = all_prompt + wrong_code + error_msg
4     answer_text = call_llm_test.call_llm(prompt_with_feedback,

```

```

        llm)
5      ans_code = parse_output.parse_generated_code(answer_text)
6
7      if not ans_code:
8          wrong_code = ""
9          error_msg = (
10              "\nCode must be returned inside a markdown code
              block, for example:\n"
11              "``python\n"
12              "def process_data(dataframes_dict):\n"
13              "    ... \n"
14              "``"
15          )
16          logger.warning("No code block generated on attempt %s",
                          attempt_index + 1)
17          continue
18
19      try:
20          result = _execute_generated_code(ans_code, data_dict)
21          assert_result = assert_func(result)
22          if assert_result:
23              raise ValueError(assert_result)
24          return result, attempts_used, all_prompt
25      except Exception as exc:
26          wrong_code, error_msg = _build_execution_feedback(exc,
27                                                         ans_code, data)
28          logger.warning(
29              "Generated code failed on attempt %s/%s with error:
              %s",
30              attempt_index + 1,
31              max_retries + 1,
32              exc,
33          )

```

Listing 4 Agent 基于异常捕获的代码重试闭环逻辑

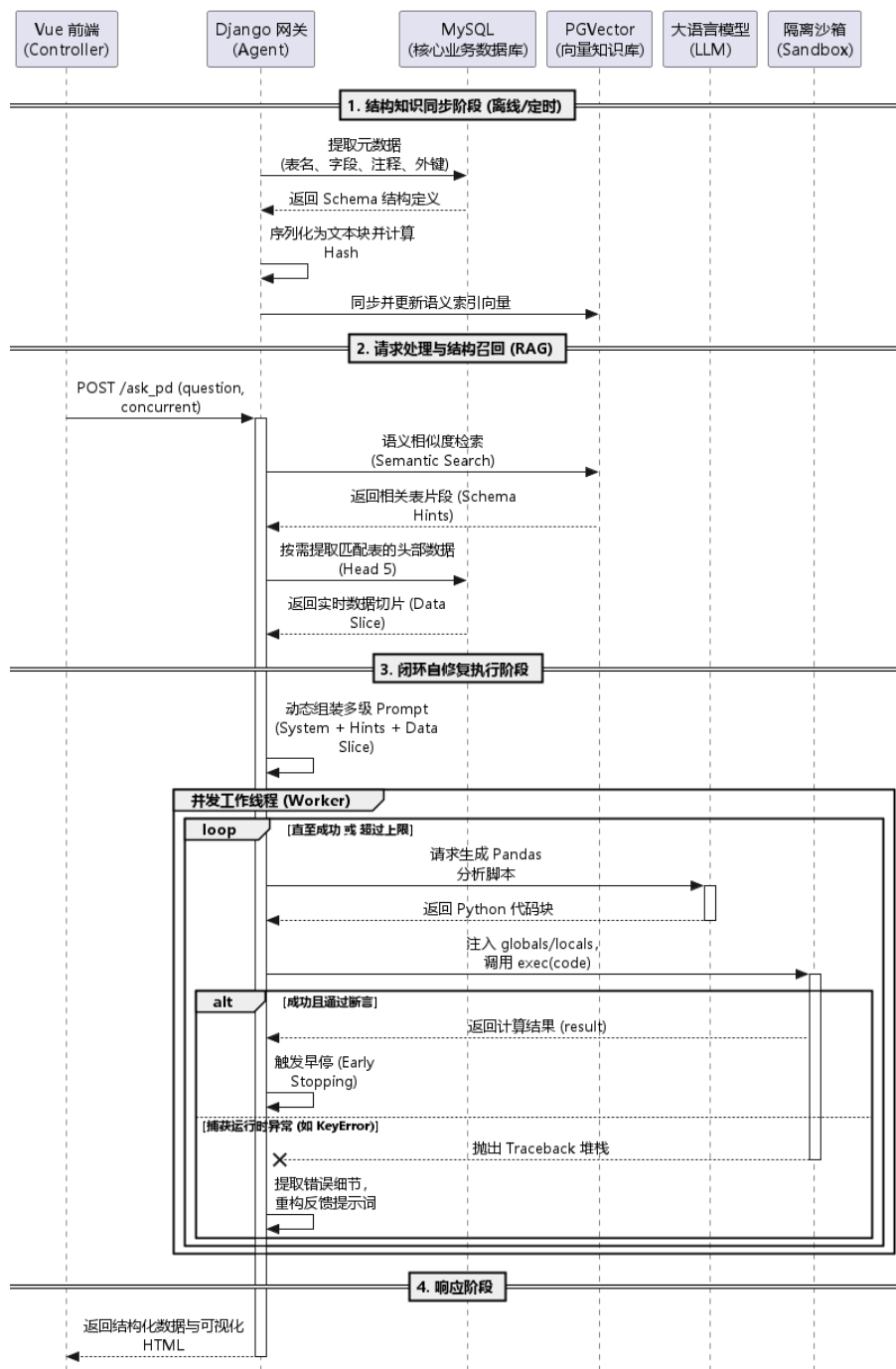


图 3.2 Agent 异常回输与闭环自修复状态机图

Fig. 3.2 Agent State Machine Diagram for Abnormal Feedback and Closed-Loop Self-Healing

3.4.2 针对 Pandas 运行时异常的语境重构

平时在做数据分析任务的时候，最容易弹出来的报错大概率就是找不到列名的 `KeyError`，或者是数据类型不支持从而引发的 `TypeError` 了；等到沙箱里调用 `exec` 函数抛出了这类的异常时，系统底层就会专门去把报错栈给拦下来，并且把

当前能够正常使用的列名集合全附在报错信息的后面。这种既指出了“错在哪”，又给出了“正确参考字典”的精准引导，能让大模型很快反应过来并且把拼写错的列名改好，保障了后续的数据处理能够顺畅跑通。

3.5 多代理并发调度与早停机制

在实际去落地大模型应用的时候，单次调接口去跑网络传输和推理的时间成本还是相当高的；如果系统在处理复杂的数据分析请求时只能靠着单线程慢慢去报错重试，那只要模型多写错几次代码，整个系统的响应时间就会被拉得很长，弄不好就会引发前端请求的超时报错。为了从根本上突破这个性能瓶颈，本系统在重构后端网关的时候，就引入了基于线程池的多代理并发调度架构。

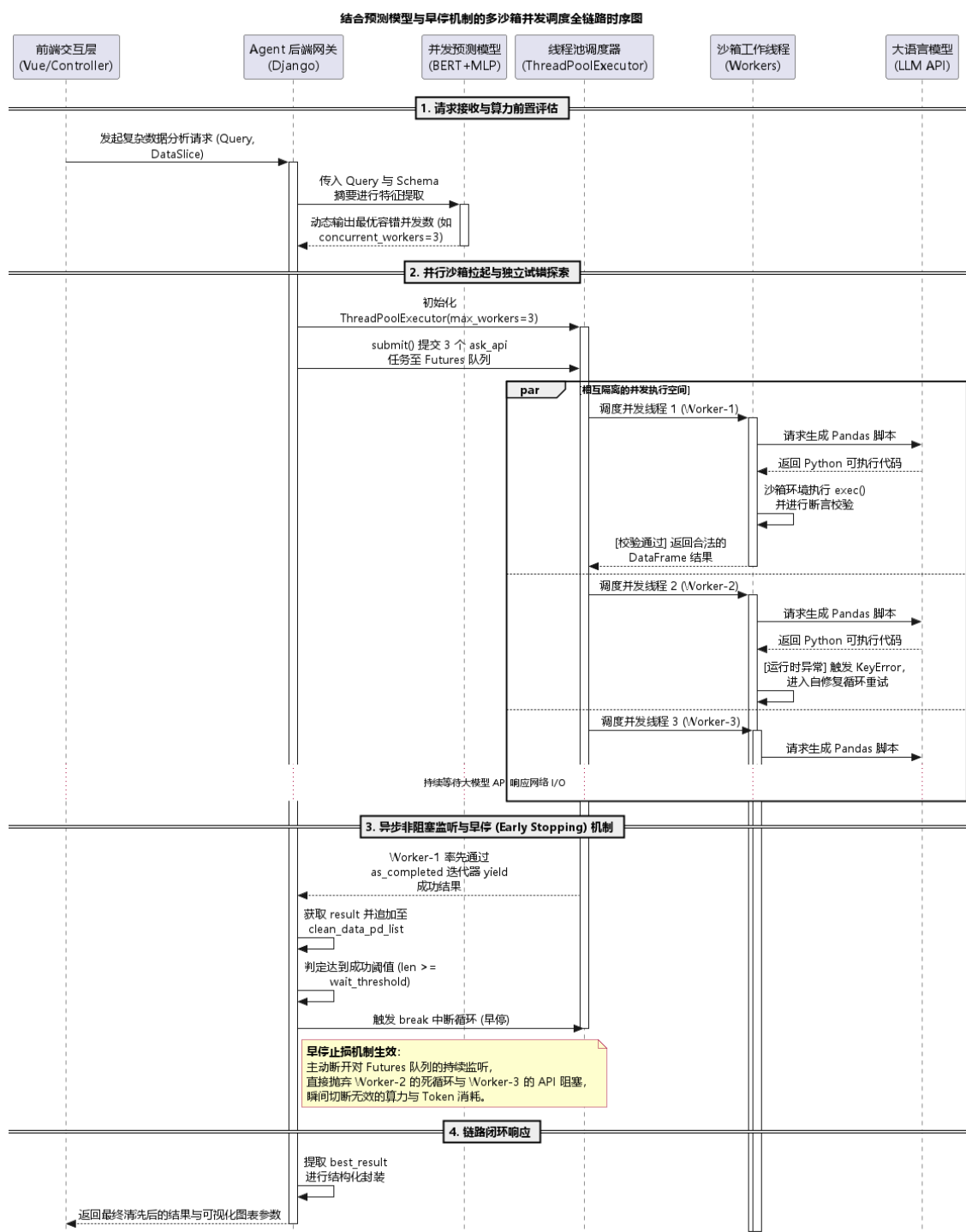


图 3.3 结合预测模型与早停机制的多沙箱并发调度全链路时序图
Fig. 3.3 Full-link Sequence Diagram of Multi-sandbox Concurrency Scheduling Combining Prediction Model and Early Stopping

考虑到去调大模型接口和用沙箱去运行代码这些事本质上都属于很占用 I/O 资源的任务，系统就在相关的模块里用上了 Python 的 ThreadPoolExecutor 去搭建了一个多路并发的线程池，这算是有意绕开了全局解释器锁对多线程性能的拖

累；只要后端接到前面评估模块算出来的推荐并发数量，就会同时创建好几个互不干扰的智能体线程，让它们各自在物理隔离的沙箱里同步去生成代码和试错。

为了不浪费算力资源，如图 3.3 所示，系统利用了 `as_completed` 迭代器非阻塞的监听特性，把经典的“早停”机制给做了出来；只要系统监控到有某一个工作线程率先跑通了数据校验并且拿到有效结果了，主线程立刻就会把这个结果采纳过来，然后稳妥地把其他还在排队或正在运行的冗余线程给掐断：

```
1 def ask_pd(data, req, llm):
2     """Generate pandas transformation results concurrently and
3         return a validated result.
4
5     Args:
6         data (list): Input payload consumed by downstream prompt
7                     assembly and execution.
8         req (Any): Request object containing 'concurrent' and '
9                 retries' settings.
10        llm (Any): Language model used for code generation.
11
12    Returns:
13        tuple: '(result_df, retries_used, all_prompt, success_ratio
14              )' where 'result_df' is 'None' on failure.
15    """
16
17    max_rounds = __safe_int(config_data["ai"].get("tries"), default
18                            =1, minimum=1)
19    wait_threshold = __safe_int(config_data["ai"].get("wait"),
20                                default=1, minimum=1)
21    concurrent_workers = __safe_int(getattr(req, "concurrent", 1),
22                                    default=1, minimum=1)
23    retries = __safe_int(getattr(req, "retries", 0), default=0,
24                          minimum=0)
25
26    prompt = get_ask_pd_prompt(req)
27    last_retries_used = 0
28    last_prompt = ""
29
30    for round_index in range(max_rounds):
```

```

22     clean_data_pd_list = []
23     with concurrent.futures.ThreadPoolExecutor(max_workers=
24         concurrent_workers) as executor:
25         futures = [
26             executor.submit(
27                 ask_api.ask,
28                 data,
29                 prompt,
30                 llm,
31                 parse_output.assert_pd,
32                 retries,
33             )
34             for _ in range(concurrent_workers)
35         ]
36
37     for future in concurrent.futures.as_completed(futures):
38         try:
39             result, retries_used, all_prompt = future.
40                 result()
41             last_retries_used = retries_used
42             last_prompt = all_prompt
43         except Exception as exc:
44             logger.exception("PD generation worker failed:
45                 %s", exc)
46             continue
47
48         if result is not None:
49             clean_data_pd_list.append((result, retries_used
50                 , all_prompt))
51             if len(clean_data_pd_list) >= wait_threshold:
52                 break
53
54 if clean_data_pd_list:
55     best_result, best_retries_used, best_prompt =
56         clean_data_pd_list[0]
57     success_ratio = len(clean_data_pd_list) / float(

```

```

53         concurrent_workers)
54     return best_result, best_retries_used, best_prompt,
        success_ratio
55     logger.warning("PD generation round %s/%s failed",
        round_index + 1, max_rounds)
56
57     return None, last_retries_used, last_prompt, 0.0

```

Listing 5 基于 ThreadPoolExecutor 的多代理并发执行与早停调度

要是从系统工程优化的角度来看，这种一边多路探索一边又懂得见好就收的调度方法，其实就是使用了拿算力空间去换取时间低延迟的策略；它一方面降低了大模型单次生成可能会发生错误的概率风险，另一方面也没有增加用户的等待时间，反倒是把系统的容错率拉高很多，为整个基于 Agent 的数据交互底座提供了扎实的工程保障。

第 4 章 基于 BERT 的查询并发预测与调度模型设计

之前的第三章曾提到，引入多路并发和早停机制确实能让系统在报错时多几次重试的机会，容错率有了不小的提升。不过这里其实还遗留了一个调度上的问题，那就是在面对难度不同的自然语言查询时，到底该给它分配多少个并发线程才算合理。为了解决这个痛点，本章主要探讨的是怎么去构建一个基于 BERT 的查询复杂度预测模型；思路就是争取在任务刚开始跑的时候，就能大概预估出需要用掉多少并发资源，从而把系统的算力分配得更合理一些。

4.1 存在问题

平时大语言模型在处理那些比较复杂的查询想法时，因为对上下文的理解能力有限，所以单次生成代码到底能不能成功，其实是表现出一定的随机性的。在真实的业务场景里，如果只用那种固定不变的静态并发配置，很容易就会碰到“资源和效果不能两全”的矛盾：一方面，要是全局并发数设得太小（比如保守一点设成 $N = 1$ ），遇到那种比较复杂的业务分析任务时，系统往往会因为一次没推导出来就直接报错中断了，系统的可用性也就跟着降下来了；另一方面，要是为了强行拔高成功率就把并发数拉得很高（比如硬性设定 $N = 5$ ），虽然确实能靠着反复试错把通过率提上去，可一旦遇到那种只是查个单表聚合的简单任务，多出来的那些并发线程就会白白浪费掉计算资源，甚至还会因为增加了系统的 I/O 负担，反倒把整体的响应速度给拖慢了。因此，那种固定的调度策略是很难去适应难度各异的查询任务的。

4.2 研究动机

数据库查询任务能不能顺利生成正确代码，主要还是取决于用户的自然语言问题跟底层表结构到底有多难匹配。要是用户发来的指令里包含了好几张表相互关联、藏着隐式条件或者是那种极其复杂的业务逻辑，模型正确生成代码的概率自然就会下降。

本研究认为，这种查询任务的“执行难度”其实是可以提前计算出来的。通过把用户输入的指令和数据库结构的上下文抽取出来做一个特征分析，系统完全可以在大模型动手写代码之前，就预估出大概需要重试几次才能成功。既然 BERT 模型在自然语言处理这块一直很擅长做特征提取和文本理解，所以本课题就把轻量的 BERT 模型引入作为评估模型，希望能搭建一个端到端的“查询并发预测模

型”，真正把系统算力资源的动态调配给建立起来。

4.3 并发预测模型设计

为了让算力预测和实际跑业务这两件事互不干扰，系统特意把并发预测模型拆出来放进了一个独立的服务模块里。这个模型整体上是由两部分拼起来的：前面是一个靠 BERT 来提取语义特征的模块，后面接着一个用多层感知机（MLP）来做回归输出的预测层。

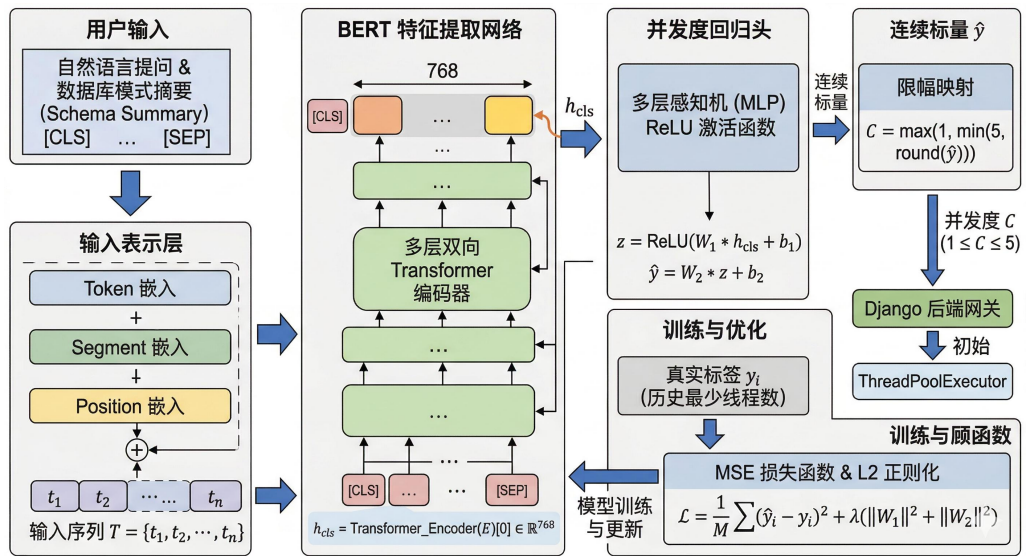


图 4.1 基于 BERT 的并发预测模型整体架构图

Fig. 4.1 Overall Architecture of BERT-based Concurrency Prediction Model

结合图 4.1 来看，预测模型拿到用户输入的自然语言指令和相关的表结构摘要之后，会让它们通过深度神经网络去处理一下，最后产生出一个具体的数值结果，用来代表系统推荐分配的并发数。为了把这个运作机制讲得更加透彻，接下来会把输入处理、特征提取还有回归预测这三个核心环节拆开来详细说明一下。

1. **输入序列重组与编码。**并发预测模型既得看懂用户的“查询意图”，又得明白底层数据库的“结构上下文”。因此系统最开始会把前端传过来的自然语言问题，跟向量知识库找回来的那些表结构片段，直接在字符串层面上拼接起来。为了对齐 BERT 模型的标准输入格式，系统会在序列的最前头塞进一个特殊的分类标记 [CLS]，然后在问题和表结构之间再插一个分隔标记 [SEP]。等分词器把这些文本处理完之后，它们会被映射成词嵌入向量，然后再叠加位置编码；这么一套流程走完，原本非结构化的文本就被转换成了包含语义信息与绝对位置顺序的高维张量。

2. **基于 BERT 的双向语义特征提取。**重组好的高维向量紧接着就会被送进

BERT 那一套多层双向 Transformer 编码器里。跟以前那种单向读取的语言模型不太一样，BERT 内部的多头自注意力机制能让每一个词在计算的时候，动态地去跟上下文里的其他词做特征交互。这就意味着模型在评估难度的时候，不仅能认出“统计”、“算个平均值”这种基础意图，还能顺着后面的表字段注释，敏锐地察觉到那些跨表连接或者复杂时间过滤的逻辑难度。等在深层网络里走完一圈前向传播，排在最开头的 [CLS] 标记就已经把全局的语义特征和任务难度都融合进去了。所以系统会直接去提取这个标记在最后一层输出的隐藏状态向量，把它当成整个查询任务难度的一个最核心的特征表示。

3. MLP 回归预测与数值映射。为了把这个高维度的特征向量变成能指导并发调度的具体参数，系统在 BERT 提取完特征后又接入了一个带着非线性激活函数（比如 ReLU）的多层感知机网络。第一层隐藏层主要是负责把特征再往下压缩和提纯，把没用的冗余信息给过滤掉；到了第二层输出层，就会把降维后的特征直接映射成一个连续的预测标量。不过在实际写代码落地的时候，底层的线程池要的是实打实的整数，而且服务器的 CPU 资源是有物理极限的；所以系统会把这个算出来的连续数值做个四舍五入，并且严格限制在一个比较安全的区间里（比如 1 到 5 之间），最终生成一个合法的并发控制参数 C 。这一连串的计算其实就是把自然语言文本一路转换成了系统下达调度的离散指令。

4.3.1 向量化特征提取机制

在正式送进模型之前，系统会把用户的提问和表结构信息拼在一起，并在开头加上 [CLS] 标记，中间用 [SEP] 标记作区分。假如把处理完的这串序列记作 $T = \{t_1, t_2, \dots, t_n\}$ 。

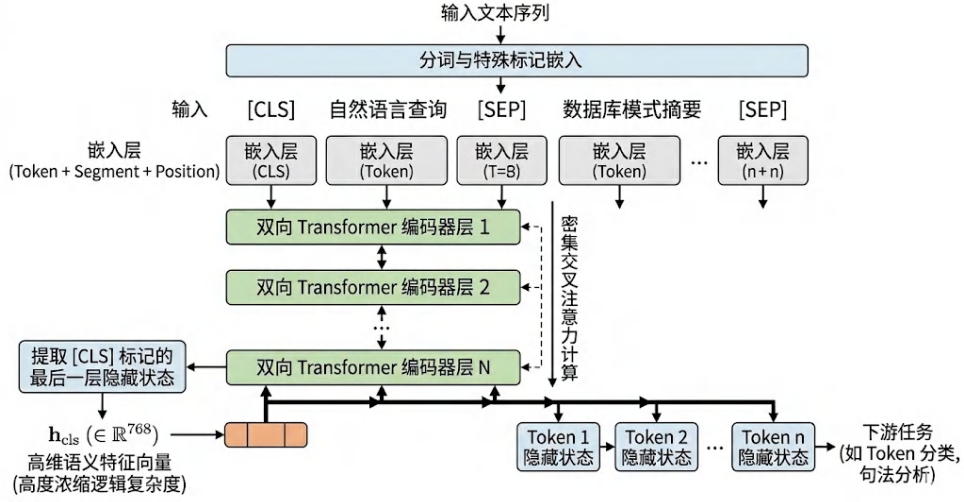


图 4.2 BERT 特征提取数据流图

Fig. 4.2 Data Flow Diagram of BERT Feature Extraction

就像图 4.2 展示的那样，输进去的文本序列变成词向量后就进入了 BERT 的编码器。靠着自注意力机制，模型能很准确地抓住句子前后的逻辑关系。系统专门把 [CLS] 标记在最后一层输出的高维向量 h_{cls} 给提取了出来；因为这个向量基本把整个输入序列的全局语义都浓缩在一起了，它能非常直观地反映出当前查询任务到底有多复杂，所以拿它来作为后续预测的基础特征是非常合适的：

$$h_{cls} = \text{Transformer}_{\text{Encoder}}(T)[0] \quad (4-1)$$

4.3.2 并发度回归层设计与目标函数

拿到特征向量 h_{cls} 之后，系统会把它传给一个包含 ReLU 激活函数的全连接网络做降维处理。第一层主要去深挖特征，第二层则负责把高维向量映射成一个可以连续变化的预测数值 $\hat{y}$ 。它的前向传播算法大概是这样的：

$$z = \text{ReLU}(W_1 h_{cls} + b_1) \quad (4-2)$$

$$\hat{y} = W_2 z + b_2 \quad (4-3)$$

在这里， W_1, W_2 分别是两层网络的权重矩阵，而 b_1, b_2 则是偏置项。

在训练模型的时候，那些真实的标准答案 y_i 其实是从系统前期跑出来的历史日志里挖掘出来的，具体定义就是“成功跑通这个查询任务最少需要几个线程”。因为预测的目标值是个连续数值，所以模型选了均方误差（MSE）来作为损失函

数，顺带还加了一个 L_2 正则化项，主要是为了防止模型出现过拟合，希望能提升一下模型在处理新数据时的泛化能力：

$$\mathcal{L} = \frac{1}{M} \sum_{i=1}^M (\hat{y}_i - y_i)^2 + \lambda (\|W_1\|_2^2 + \|W_2\|_2^2) \quad (4-4)$$

在这个公式里， M 是每次打包训练的样本数， λ 是正则化系数。通过反向传播算法一遍遍去修正网络权重，最终目的就是让预测出来的 $\hat{y}$ 越接近真实并发度越好。

4.4 动态调度策略实现

前面提到 BERT 模型最后算出来的 $\hat{y}$ 其实是个带有小数的浮点数，无法直接塞给线程池当初始化参数用。而且考虑到服务器的 CPU 和内存不可能无限往上加，还得给这个预测结果设定一个合理的边界。系统就是用下面这个公式把那个连续预测值变成整数并发度 C 的：

$$C = \max(1, \min(5, \lfloor \hat{y} \rfloor)) \quad (4-5)$$

公式里的 $\lfloor \cdot \rfloor$ 代表的就是四舍五入取整操作。等算出了并发度 C 以后，系统就会把这个数值传给后端的业务网关；网关再根据这个数去动态设定 `ThreadPoolExecutor` 的最大工作线程数。有了这个机制，再配合上系统内部的早停操作，系统就能根据查询任务的难易程度分配合适的计算资源，规避掉了固定开高并发造成的资源浪费。

4.5 实验设计与分析

为了验证前面提出来的这套 BERT 并发预测模型在实际跑业务时到底有没有效果，本节基于独立准备的 100 组测试用例，用传统的静态并发策略与动态预测策略做了一次比较系统的量化对比。

4.5.1 实验环境与数据集构建

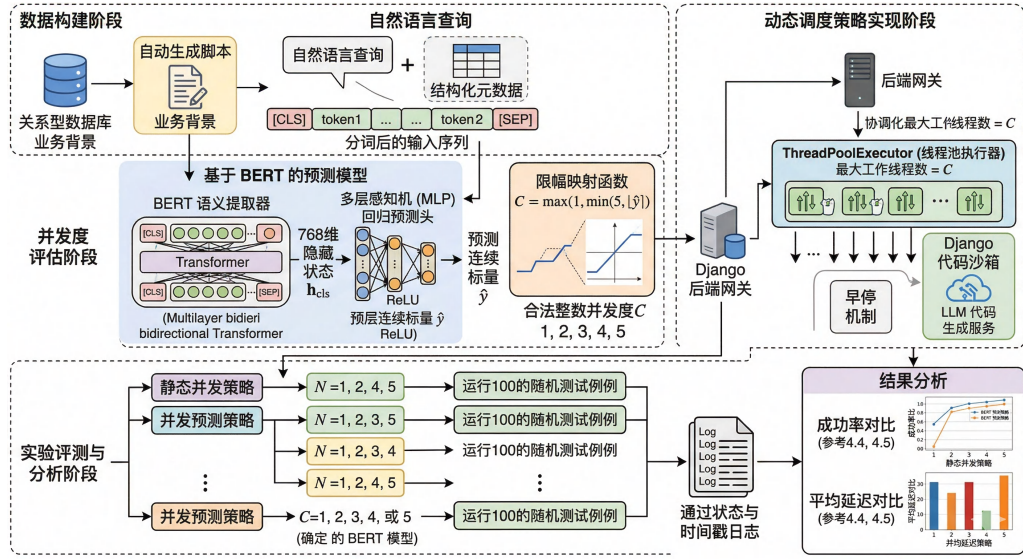


图 4.3 实验数据流转架构图

Fig. 4.3 Experimental Data Flow Architecture Diagram

如图 4.3 所示，实验准备的这 100 组测试用例涵盖了多种多样的场景，包括单表过滤、多表联查以及聚合计算等自然语言指令。为了防止外界因素干扰 API 的响应速度，所有的实验都是在完全相同的硬件环境和网络条件下按顺序去跑的。

实验评价体系主要看两项核心指标：第一项是任务成功率 (Success Rate)，主要看系统最后到底能返回几个合规的 DataFrame 数据；第二项是平均延迟 (Average Latency)，就是去统计从任务发下去，一直到沙箱返回结果或者触发早停，总共花了多长时间。实验的对照组用的是那种固定数值的静态线程池策略，也就是分别测试 $N \in \{1, 2, 3, 4, 5\}$ 的情况；而实验组用的则是基于 BERT 模型算出来的动态调度策略。

4.5.2 并发度与任务成功率的边际效用分析

首先来观察在不同的静态并发度 (1-5) 和使用并发预测调度模型时，大模型推理成功率的变化。

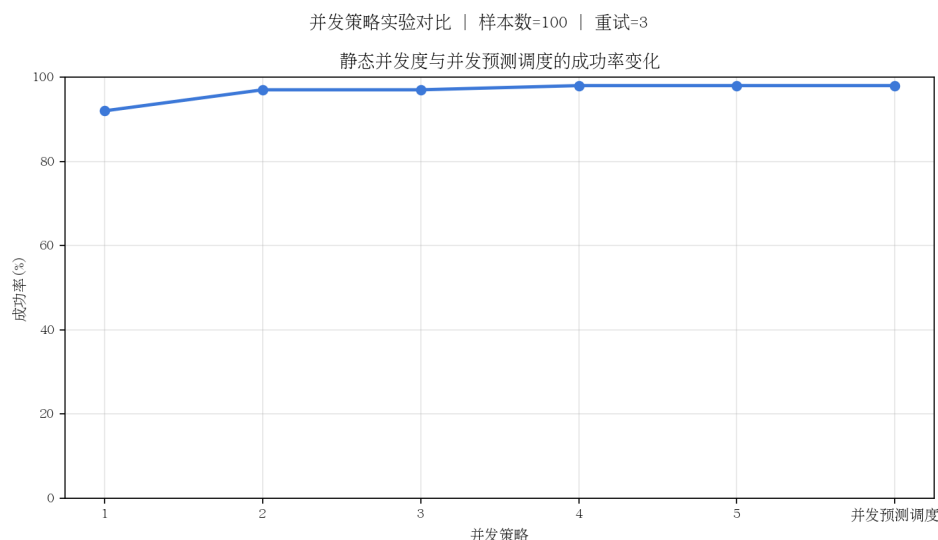


图 4.4 不同静态并发度与并发预测下的成功率变化折线图

Fig. 4.4 Line Chart of Success Rate Changes under Different Static Concurrency Levels and Concurrency Prediction

图 4.4 和测试数据反映出一个现象：大模型单次推理其实有着挺不错的基础可用性，就算只开一个单线程 ($N = 1$)，成功率也达到了 92.0% 的及格线。把并发数往上提，加到 $N = 2$ 和 $N = 3$ 的时候，成功率就涨到了 97.0%；等并发度继续增加到 $N = 4$ 时，成功率达到了 98.0%。

不过数据也揭示了一个边际收益递减的问题：哪怕系统把并发数拉满到上限 $N = 5$ ，系统的成功率依旧是停留在 98.0% 没有再往上增加。相比之下，当系统切换到 BERT 并发预测策略后，整体的成功率依然能够稳稳地收敛在 98.0% 这个高水位区间上；这就初步证明了，加了这个预测模型之后，系统在保障任务能顺利跑完这方面没有出现性能倒退。

4.5.3 预测调度在延迟开销上的置换效益评估

既然搞清楚了成功率的上限，这一小节就接着考察算力的不断扩张会给系统的等待时间带来多少负面代价。

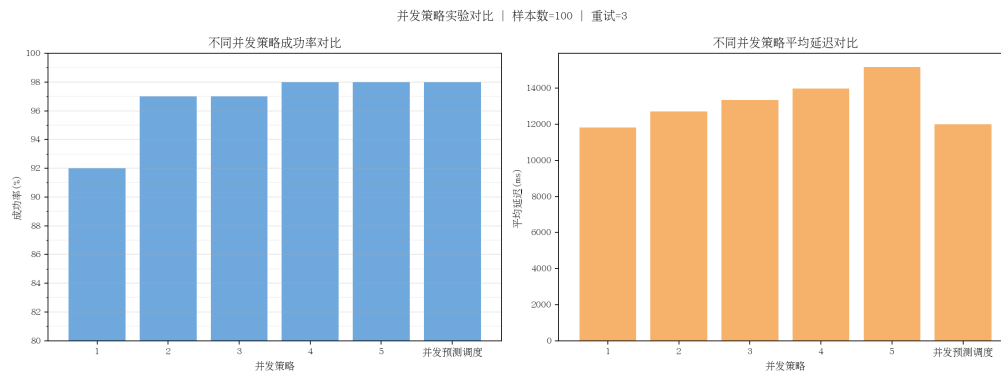


图 4.5 不同静态并发度与并发预测下的成功率与平均延迟对比柱状图

Fig. 4.5 Bar Chart Comparison of Success Rate and Average Latency under Different Static Concurrency Levels and Concurrency Prediction

从图 4.5 就能看出来，静态开高并发的做法基本是用执行效率去换成功率。测试数据显示， $N = 1$ 的时候系统平均要等待大约 11830 毫秒；但随着线程数线性的逐渐增加，底层抢 I/O 的现象和大模型端点的排队耗时明显变得严重起来。等到用了 $N = 5$ 这个最高并发的时候，平均等待时间直接升到了 15180 毫秒，增幅将近 28.3%。这也说明了，面对绝大多数的简单任务时，静态高并发策略平白无故引入了太多没必要的算力空转和进程调度负担。

反过来再看基于 BERT 的预测调度机制，它在资源的自适应分配上就显得很有优势。因为这个模型能够准确地从 100 组指令里把那些很简单的基准查询给过滤出来，然后只分配少量的容错线程去运行；只有在识别到嵌套得很深的复杂任务时，才会把高并发给开起来。靠着这种动态调度的手段，总体的平均延迟被拉回了接近低并发调度的基准区间里。实验结果证明，这个调度架构在保证 98.0% 峰值成功率的前提下，成功避免了全局高并发引发的系统卡顿问题，在可用性和响应速度之间找到了很好的平衡。

4.6 本章小结

针对大语言模型在生成代码时成功率经常波动的问题，本章提出并实现了一套基于 BERT 的查询并发预测方案。大致思路就是利用 BERT 模型去抓取自然语言指令和数据库结构的语义特征，然后再接入全连接神经网络，让系统能在运行查询任务之前就预估出开几个并发线程比较合适。最后的实验结果也表明了，与以前那种死板的静态固定并发比起来，动态调度的策略在保住高任务成功率的同时，减少了很多没必要的试错操作，让系统的平均响应时间也下降不少。这为后续提升智能查询系统的实际运行效率和算力利用率提供了一个确实可行的方法。

第 5 章 数据库高效查询原型系统设计与实现

前面的第三章和第四章已经分别讨论了大语言模型的代码生成自修复机制以及智能并发调度策略，本章主要介绍数据库高效查询系统在工程上的具体实现过程。在开发阶段，本系统采用了前后端分离与微服务架构，将负责界面的前端、处理核心业务的后端网关以及运行模型的算法模块分离开来。通过这种设计，最终搭建出了一个可以通过自然语言直接查询底层数据库的交互平台。

5.1 系统需求分析

系统开发的首要环节在于清晰界定需求，这直接关系到最终产出的实际应用价值。为了切实缓解企业在复杂业务场景下面临的数据获取难题，在正式进入编码阶段前，非常有必要对不同业务岗位的真实诉求进行深入调研，从而精准划定系统的功能边界。

5.1.1 系统角色分析

这一系统所面向的受众群体主要能划分为两大核心类别：一是那些身处业务一线的分析人员与管理人员，其在日常开展工作时感到最为不便的往往是去编写那些逻辑极其复杂的 SQL 语句，所以能够直接通过自然语言来检索数据并生成对应的可视化报表就成为了这一类群体最为迫切的实际诉求；二是负责维护系统的管理员或者是数据库管理员（DBA），由于系统把底层的知识库向量化同步设计成了能够随着用户查询而自动触发的静默加载模式，从而显著地减轻了后台的运维工作量，进而使得管理员能够把主要的精力投入到保障物理数据库节点运行的安全与可靠性上。

5.1.2 功能性需求与成文用例分析

把前述这两类受众的实际需求给结合起来看的话，系统的核心功能需求就能被梳理为三条逻辑主轴，为了能更直观地把系统内部数据流转的底层细节给呈现出来，本小节专门把相关的用例图给绘制好了，并搭配了标准的成文用例表单去对各项核心功能进行深度地剖析。

(1) 自然语言数据查询（含隐式知识库同步）用例

第一点是自然语言数据查询功能，这被认为是整套系统里最具业务价值的核心能力，它能让那些完全没有数据库语法背景的业务人员也可以迅速地把结构化

数据给获取到手；并且非常关键的一点在于，为了确保大模型在每一次进行逻辑推演时都能精准地参考最新的表结构上下文，查询链路中被专门嵌入了一套隐式的知识库同步机制，也就是系统在接收前端发来的请求时，会顺带着把物理业务库给扫描一遍，并把最新的表结构给执行向量化处理，最后再静默地更新到后台的 PGVector 知识库之中。

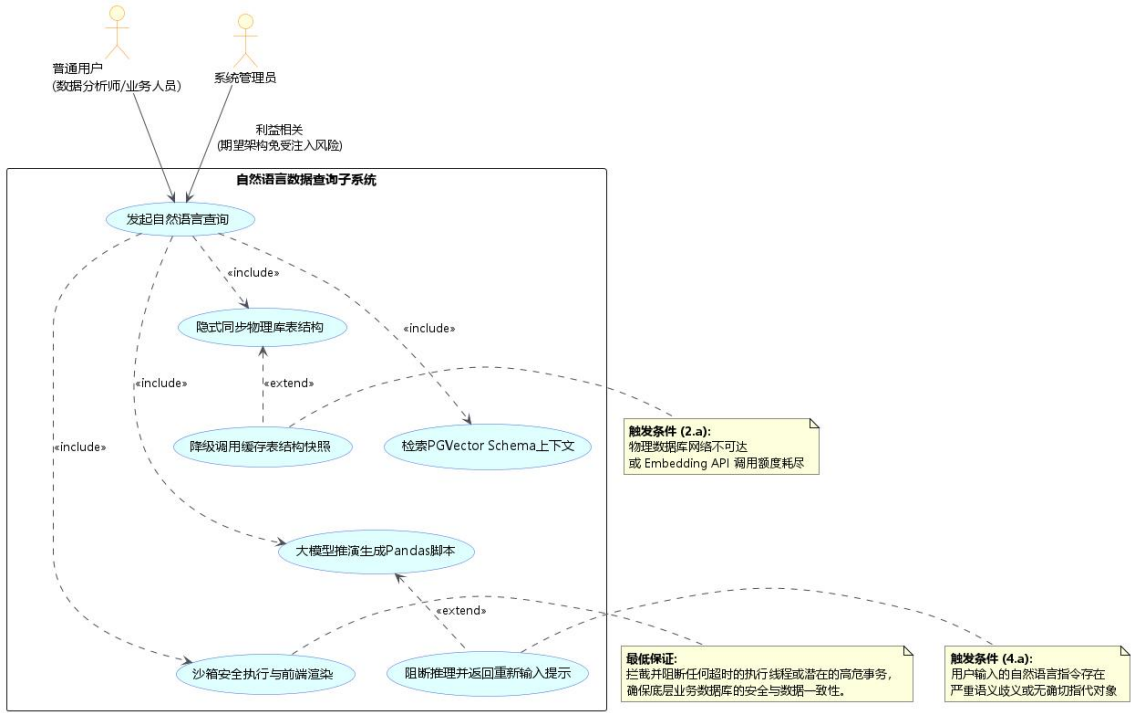


图 5.1 自然语言数据查询用例图

Fig. 5.1 5.1 Natural Language Data Query Use Case Diagram

表 5.1 自然语言数据查询成文用例

Table 5.1 Natural Language Data Query Textual Use Case

项目	内容
用例名称	自然语言数据查询（含隐式知识库同步）
主参与者	普通用户（数据分析师/业务人员）
层次	海平面（用户目标层）
利益相关者	普通用户（期望高效获取精确数据）与系统管理员（期望底层架构免受注入风险）

续下页...

表 5.1 自然语言数据查询成文用例（续）
Table 5.1 Natural Language Data Query Textual Use Case (Continued)

项目	内容
前置条件	用户已完成身份鉴权，且后端网关与物理业务数据库的连接池状态正常。
最低保证	拦截并阻断任何超时的执行线程或潜在的高危事务，确保底层业务数据库的安全与数据一致性。
成功保证	成功完成底层表结构的隐式向量化同步，精准解析用户语义，并生成合法的脚本返回二维数据集。
触发器	用户在前端交互界面的文本域中提交自然语言查询指令，并触发执行事件。
主要成功情节	1. 用户输入涵盖多表联合或条件过滤等复杂逻辑的自然语言提问。 2. 业务网关接收请求后，隐式扫描底层物理库表结构并将其向量化更新至 PGVector 检索引擎。 3. Agent 核心执行域检索 PGVector 知识库，获取与当前提问最匹配的 Schema 上下文。 4. 大语言模型基于增强后的上下文提示，成功推演并生成处理数据的 Pandas 脚本。 5. 沙箱安全环境加载并执行该脚本，最终将返回的结果集渲染为前端数据表格。
扩展	2.a 物理数据库网络不可达或 Embedding API 调用额度耗尽。 2.a.1 系统自动放弃本次知识库更新任务，降级调用内存中上一次成功缓存的表结构快照继续处理逻辑。 4.a 用户输入的自然语言指令存在严重语义歧义或无确切指代对象。 4.a.1 系统主动阻断大模型推理，向前端返回降级提示并引导用户重新规范指令结构。

(2) 智能图表动态生成用例

单纯提供二维数据表格仍然显得不够直观，因此系统还需要具备一定的数据特征感知能力。当查询返回的数据集恰好包含时序跨度或分类维度等适合进行图形化展示的特征时，前端视图会自动触发 ECharts 渲染引擎，将柱状图或折线图 等交互式图表直接呈现给用户，从而提供更适合直观展示的分析结果。

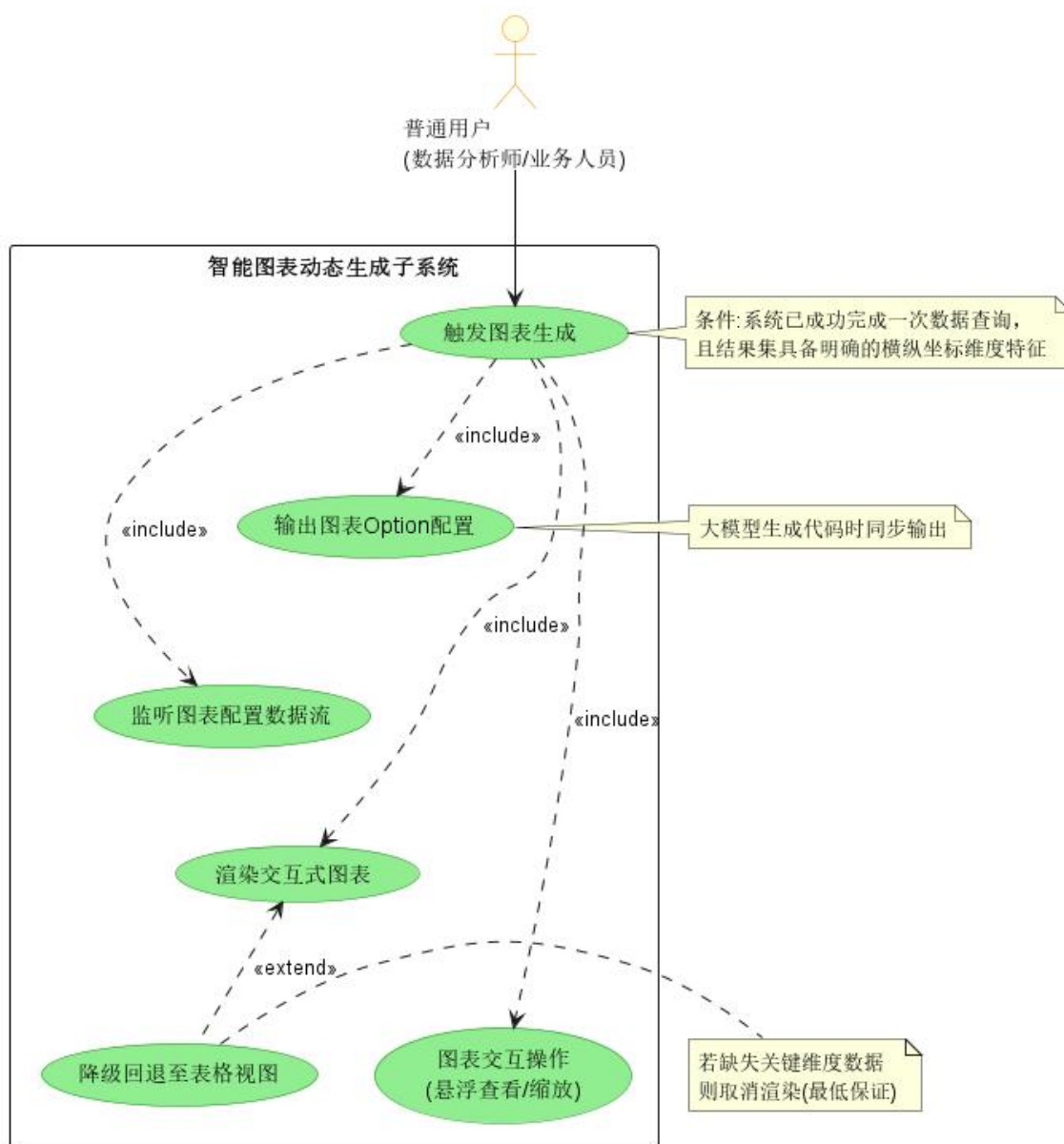


图 5.2 智能图表动态生成用例图

Fig. 5.2 Intelligent Chart Dynamic Generation Use Case Diagram

表 5.2 智能图表动态生成文用例
Table 5.2 Intelligent Chart Dynamic Generation Textual Use Case

项目	内容
用例名称	智能图表动态生成
主参与者	普通用户（数据分析师/业务人员）
层次	子功能层
利益相关者	普通用户（期望获得具备直观视觉效果的分析成果）
前置条件	系统已成功完成前置的数据查询任务，且结果数据集具备明确可映射的横纵坐标维度特征。
最低保证	若嗅探模块判定数据不具备图表转化价值，则直接取消渲染流程，降级至单纯的表格视图。
成功保证	前端展示层成功加载 ECharts 引擎，并渲染出指定类型且支持交互缩放的可视图表。
触发器	用户的原始查询意图中明确声明了图表类型，或系统解析代码结果时捕获到了图表渲染配置项。
主要成功情节	1. 大语言模型在生成数据聚合代码的同时，规范输出了图表渲染所需的 Option 配置清单。 2. 前端图表组件成功截获并解析图表配置数据流。 3. 前端界面成功装载并渲染出高保真的交互式柱状图、饼图或时序折线图。 4. 用户通过鼠标等设备在界面上对生成的图表进行数据悬浮探查与缩放交互。
扩展	1.a 大语言模型生成的图表配置项发生幻觉，遗漏了核心坐标系维度数据。 1.a.1 渲染进程在校验阶段抛出异常并主动放弃图形化操作，平滑回退至基础表格展示模式。

(3) 异常自修复感知用例

最后一个核心环节则是部署在底层的异常自修复机制。众所周知，大语言模型在生成代码时偶然的逻辑短路或字段拼写错误在所难免，为了防止这些底层报错直接抛出而破坏前端交互体验，沙箱执行层被赋予了拦截与纠错的职责：一旦监测到代码运行崩溃，执行引擎会捕获相应的异常堆栈并将其作为负向反馈重新传导给大模型，要求其在后台反复重构代码，直到顺利跑通后才将最终的正确结果推送至前端，使得整个纠错过程对终端用户完全透明。

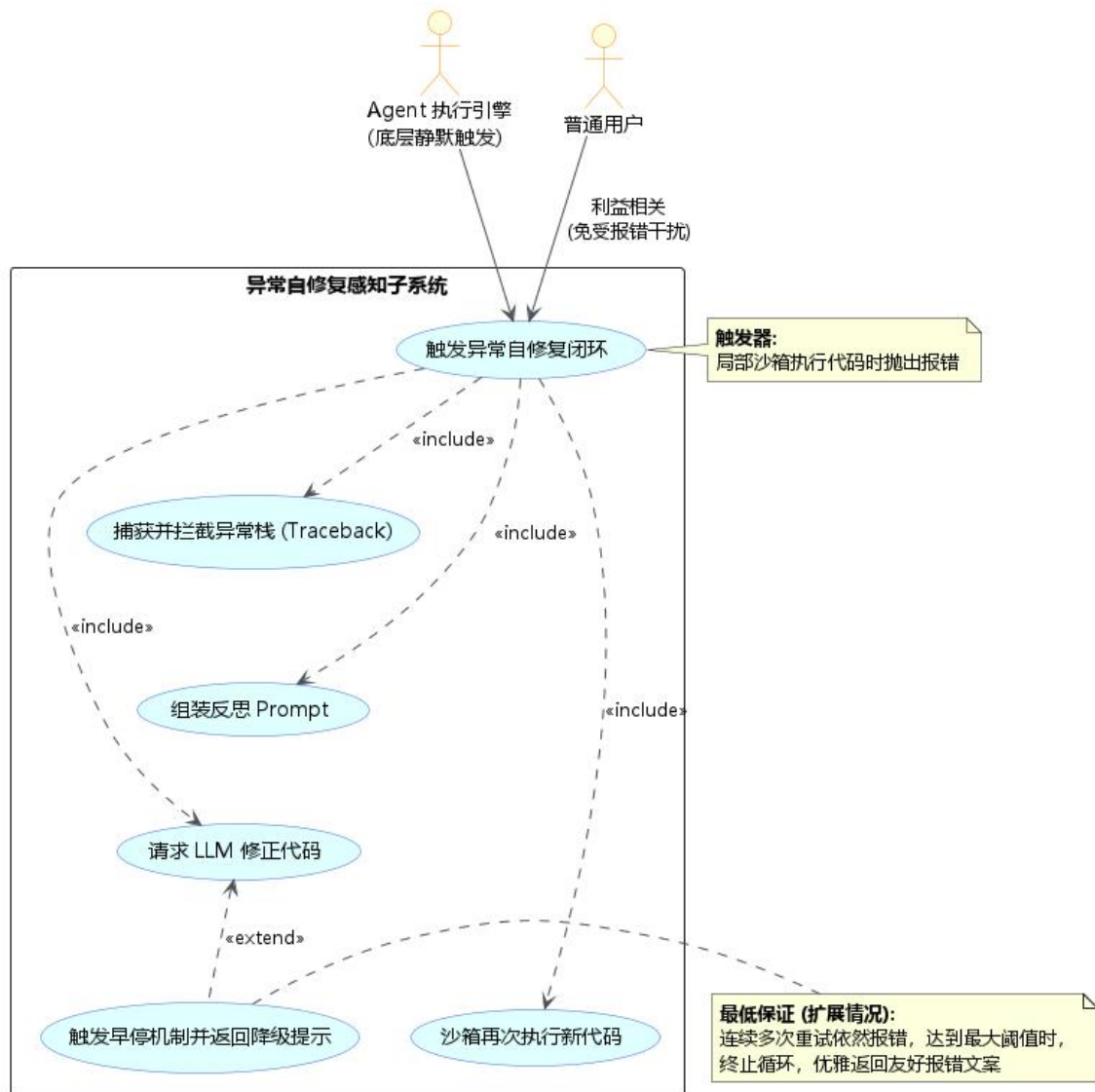


图 5.3 异常自修复感知用例图

Fig. 5.3 Anomaly Self-repair Perception Use Case Diagram

表 5.3 异常自修复感知成文用例
Table 5.3 Anomaly Self-repair Perception Textual Use Case

项目	内容
用例名称	异常自修复感知
主参与者	Agent 执行引擎（由系统底层逻辑静默触发）
层次	内部子用例层
利益相关者	普通用户（屏蔽底层报错堆栈干扰，保障流畅的业务交互体验）
前置条件	系统沙箱在执行大模型生成的 Python 操作脚本时触发了未预期的运行时异常（Runtime Error）。
最低保证	当持续重试次数触碰全局设定的安全熔断阈值时，强制终止闭环，向前端输出友好的降级容错文案。
成功保证	执行引擎通过提取异常特征栈成功引导大模型自主重构代码逻辑，并最终校验通过输出合法结果集。
触发器	代码执行沙箱（Sandbox）在编译或运行阶段抛出诸如 Key-Error 或 TypeError 等严重的执行级错误。
主要成功情节	1. Agent 引擎内的守护协程捕获并接管代码运行期抛出的异常调用栈（Traceback）。 2. 提取器截取异常核心详情，并将其与当前有效的物理表头字段重新组装为带有负向反馈的反思提示词。 3. 引擎将该结构化反馈并入新的上下文请求流，要求底层语言模型针对原代码进行自适应修正。 4. 大语言模型在认知框架内完成逻辑更迭，输出修正后的升级版操作脚本。 5. 新版本代码在安全沙箱环境中顺利通过测试与执行，内部自修复闭环正式结束。
扩展	4.a 代码经历多次反馈重构后，依然无法规避致命的编译或执行报错。 4.a.1 触发系统内置的早停（Early Stopping）机制进行最终兜底，向调用方抛出算力耗尽或查询逻辑过载的降级提示。

5.1.3 非功能性需求

为了能让原型系统从实验室环境平滑迁移至企业级的真实生产环境，并一直保持高度的健壮性，除了要实现既定的核心业务逻辑，系统在架构设计以及底层防护方面也一定要严格地满足如下非功能性的约束条件：

- **执行安全性与权限隔离：**传统的 Text-to-SQL 往往面临直接操作物理库的风险，而本系统采用的 Text-to-Python 范式虽然提升了逻辑表达的上限，但也同样引入了执行未知脚本的重大隐患。因此，系统必须在执行层构筑坚固的代码沙箱（Sandbox），严格剥离并限制 `exec()` 解释器对操作系统底层模块（如 `os`、`sys`）的访问权限，从物理层面杜绝越权读取、恶意注入以及对核心业务数据的破坏性修改（如 DDL 操作），确保整个数据库底座的绝对安全。
- **响应性能与并发调度优化：**鉴于调用远端大语言模型 API 必然伴随不可忽视的网络传输与推理延迟，如何在保障正确率的前提下压缩等待时间便成了性能设计的核心。系统不仅需要通过维护高性能的数据库连接池来应对瞬时的高频访问请求，更依赖于前期设计的 BERT 复杂度预测与早停动态调度机制，力求在算力开销与执行耗时之间取得平衡，将绝大多数复杂查询的端到端延迟控制在业务场景可容忍的阈值范围内。
- **高可用性与系统容错空间：**考虑到大语言模型固有的幻觉特征与输出不确定性，系统必须具备较强的容错与自愈能力。这要求底层架构能够从容应对 API 限流、网络波动以及代码运行期崩溃等突发状况，通过内置的异常自修复闭环将执行错误悄无声息地消化在系统内部；同时，在穷尽最大重试次数后，系统需主动阻断异常并向前端输出友好的降级提示，坚决避免将晦涩的系统级报错堆栈直接暴露给终端操作者。
- **架构扩展性与解耦部署：**随着后期业务数据字典的膨胀以及大模型并发请求的激增，单一耦合的服务器架构注定无法承载极端的计算压力。为此，系统在设计之初便深入贯彻了微服务解耦理念，将负责算力预估与模型训练的重度计算模块与处理日常请求分发的业务网关进行物理级隔离部署。这种设计使得未来可以针对预测模型的 GPU 资源单独进行横向扩容，而完全不会波及基础 Web 服务的正常流转。
- **易用性与跨端兼容性：**作为一款致力于打破技术人员与数据资产之间“鸿沟”的智能化交互平台，系统必须兼顾终端体验的流畅度与易用性。前端展示层依托 Vue 3 框架构建为纯粹的 B/S（Browser/Server）架构，使得业务人员无需在本地安装配置任何繁琐的 Python 运行环境或数据库驱动，仅需依靠主流的现代 Web 浏览器即可随时发起自然语言问答，并无缝浏览由 ECharts 引擎动态渲染的复杂数据图表。

5.2 系统架构设计

由于大语言模型存在较高的网络通信开销，同时 BERT 预测网络在进行计算时对硬件资源有一定要求，本系统在整体设计上采用了分布式与模块化的微服务架构^[21]。

具体来说，系统将整个数据查询流程分成了视图展示、业务控制、算法推理和数据存储四个独立的区域。这种架构的优点是各个模块之间互不干扰。在并发量较高的情况下，即使算法节点出现内存不足或大模型接口调用超时，这些异常也只会限制在各自的模块内部，不会导致前端页面或者核心网关崩溃，从而提高了系统整体的稳定性。

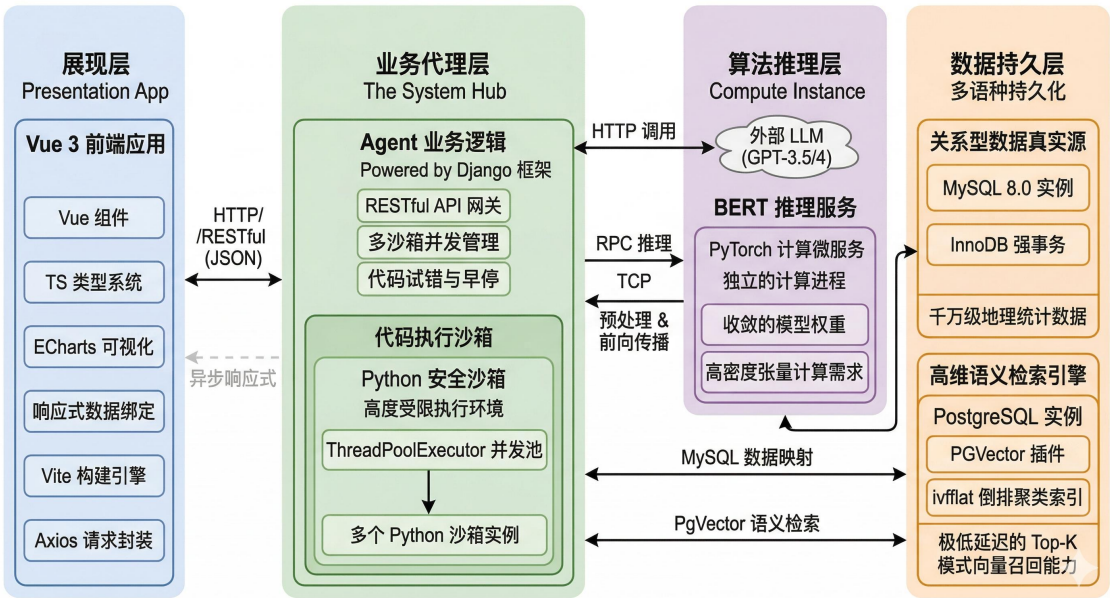


图 5.4 系统架构设计图

Fig. 5.4 System Architecture Design Diagram

5.2.1 总体微服务架构拓扑

在上述架构设计的基础上，系统将各个逻辑模块部署为物理节点。整体拓扑结构被划分为四个层级：展现层、业务代理层、算法推理层与数据持久层。

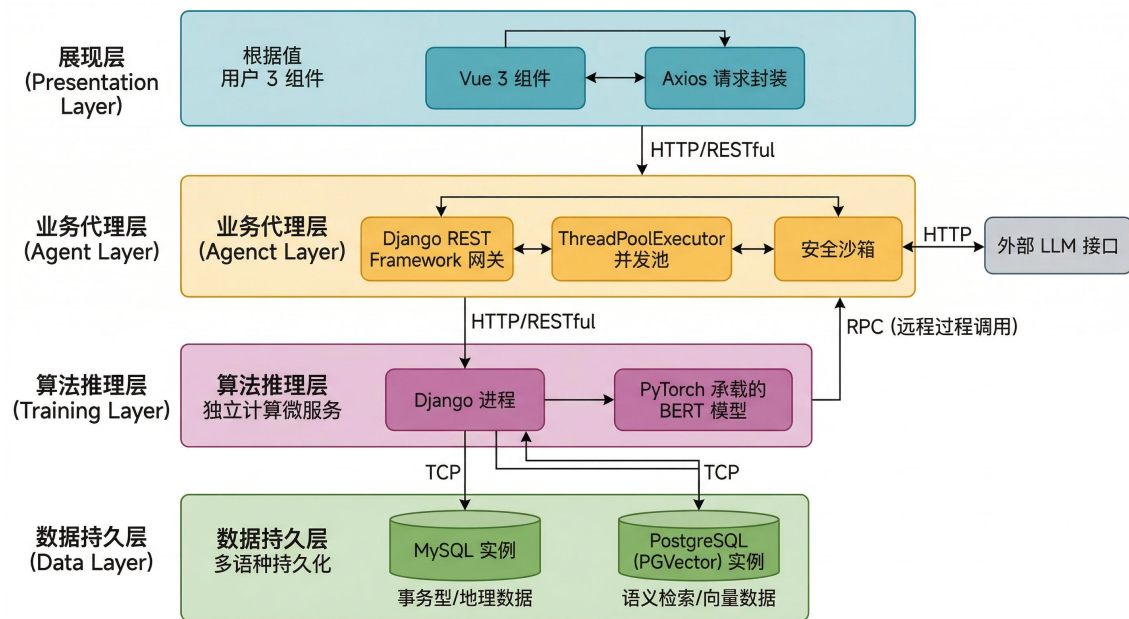


图 5.5 总体微服务架构拓扑图
Fig. 5.5 Overall Microservice Architecture Topology Diagram

展现层：该层使用了基于 Vue 3 和 TypeScript 开发的前端单页应用。通过 Vite 构建工具，前端能够按需加载图表和数据表格组件。页面内部使用了双向绑定的数据机制，在等待后端返回结果时，前端会显示骨架屏（Skeleton Screen）以提升用户的视觉体验^[22]。

业务代理层：该层是系统的核心路由，基于 Django 框架开发。这里不包含复杂的神经网络模型，主要负责处理网络请求的调度。Agent 层对外提供 RESTful API 接口接收前端数据^[23]，内部通过 HTTP 请求调用外部的大语言模型，同时负责管理 Python 代码沙箱，并利用 ThreadPoolExecutor 控制多线程并发和早停操作。

算法推理层：由于 BERT 网络在运行时依赖 PyTorch 框架并占用较多内存，系统将其抽离为一个单独的微服务。该层内部存放了训练好的模型文件，对外提供一个简单的调用接口。Agent 层将请求文本传过来后，该层能在短时间内计算出并发度参数，避免了计算任务阻塞常规的业务流。

数据持久层：在底层数据存储上，系统采用了两种不同的数据库^[24]。MySQL 8.0 作为主要的业务数据库，负责存储结构化的统计数据，并为沙箱提供数据支持；PostgreSQL 则配合 PGVector 插件作为向量数据库，利用索引功能提供表结构向量的快速相似度检索。

5.2.2 核心模块动态交互时序

为了说明用户请求在各个微服务之间的流转过程，本节通过时序图对“用户发起复杂查询”的交互流程进行了梳理。

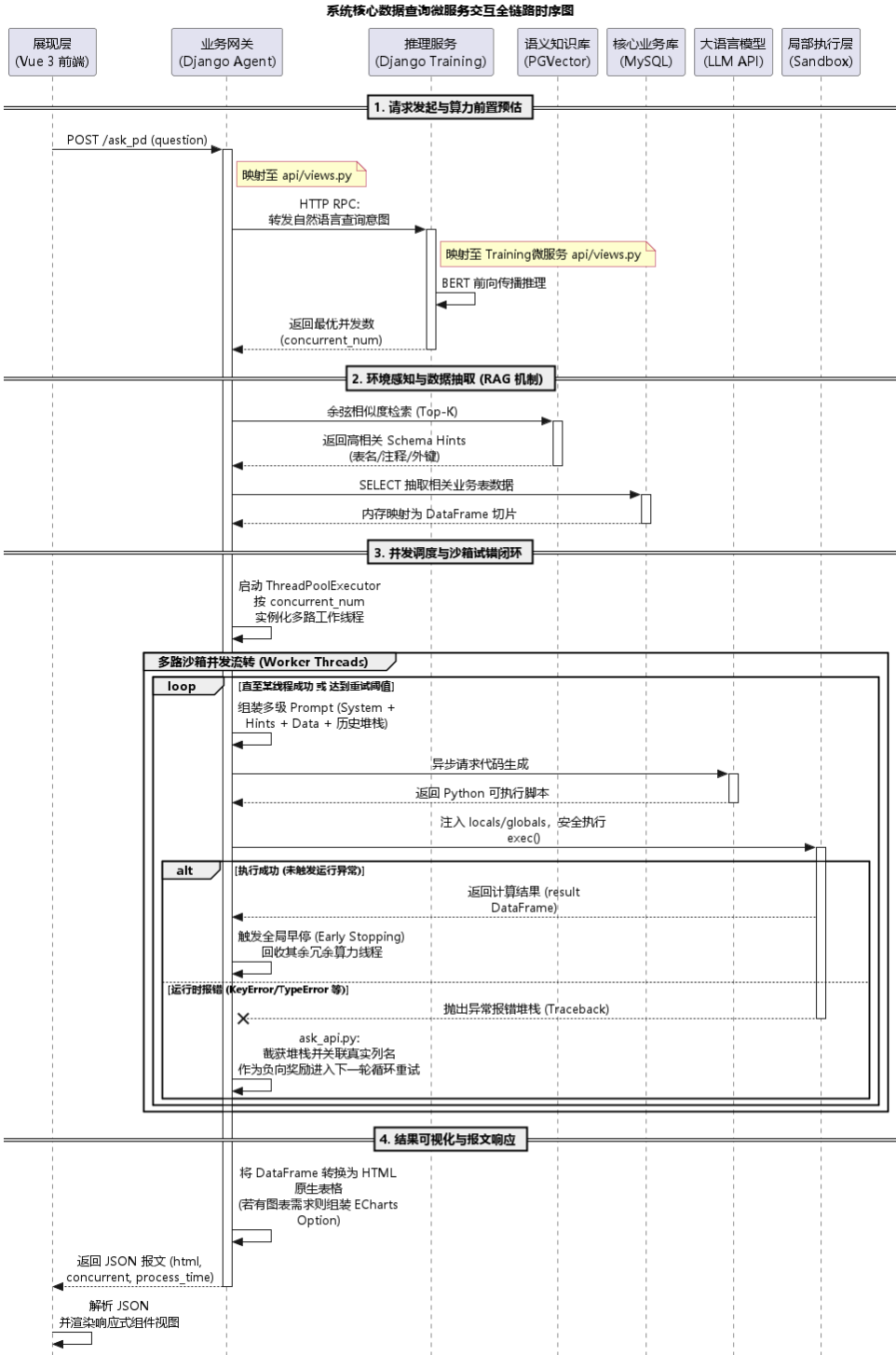


图 5.6 系统核心数据查询微服务交互全链路时序图

Fig. 5.6 Microservice Interaction Sequence Diagram for Core Data Query

当用户在前端输入问题后，请求会被发送到后端的 Agent 节点。Agent 收到请求后，先通过接口调用 Training 微服务，获取到该次查询推荐的并发参数。随后，Agent 节点内部开始执行并发 workflow：主线程一方面从 PGVector 数据库中检索相关的表结构提示，另一方面从 MySQL 中提取所需数据；接着，各个子线程携带整合好的提示词，异步向大语言模型请求生成代码。生成的代码被放入沙箱中执行试错，只要有一个线程率先执行成功并得到数据，系统就会触发早停信号终止其他线程，最后将结果组装为 JSON 格式返回给前端。

5.2.3 数据库表结构图

为了测试系统处理多表关联查询的能力，本系统采用了经典的 world 全球地理与人口统计数据集作为底层的测试数据。该数据集主要由三张表构成，符合关系型数据库的基本设计规范，减少了数据冗余。

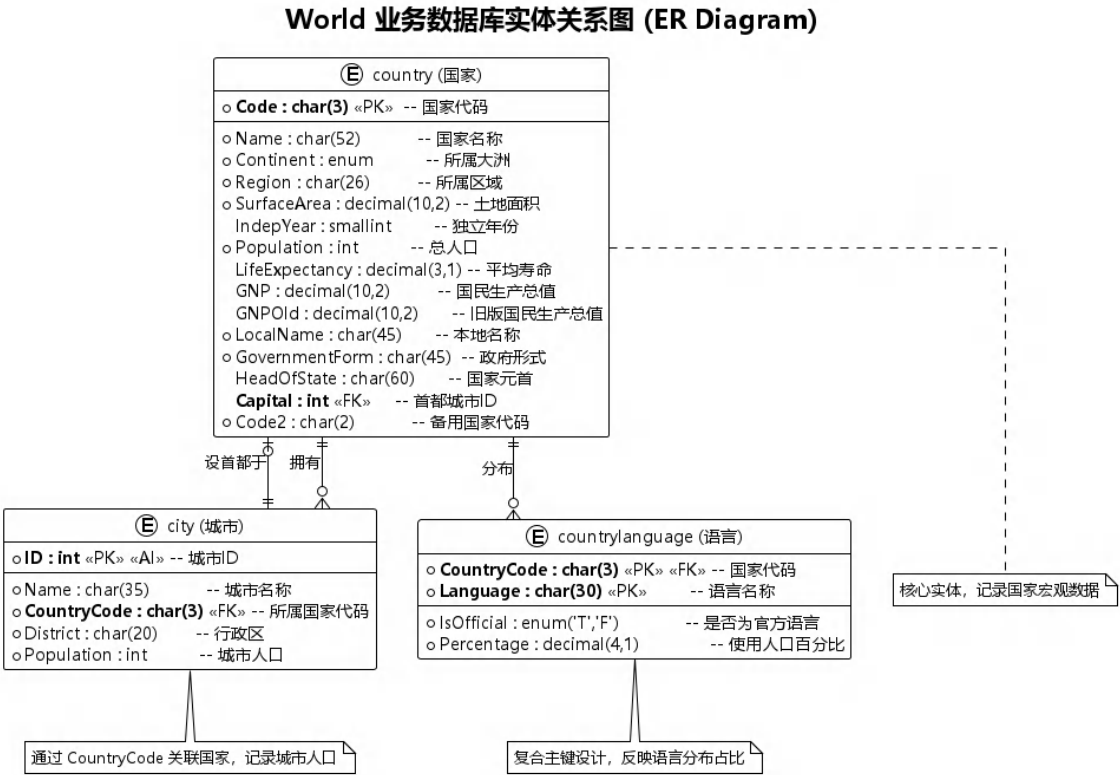


图 5.7 World 业务数据集核心实体关系图 (ER 图)

Fig. 5.7 Core Entity-Relationship Diagram of World Business Dataset

从图 5.7 中可以看出，country 表存储了国家的基本信息；city 表记录了各城市的信息，并通过 CountryCode 外键与国家表关联；countrylanguage 表记录了不同语言的分布情况，也通过外键关联到国家表。这种多表之间的关联关系，可以

用来检验大语言模型生成联合查询代码的准确性。

为了明确物理表的具体设计，以下列出了这三张核心表的字段说明。

第一部分是国家实体表（country），它的详细结构见表 5.4：

表 5.4 country (国家实体表) 表字段说明
Table 5.4 country Table Field Descriptions

字段名称 (Field Name)	数据类型 (Data Type)	约束条件 (Constraints)	业务含义说明 (Business Description)
Code	char(3)	主键 (PK)	国家的唯一三位字母代码
Name	char(52)	非空 (NOT NULL)	国家标准名称
Continent	enum	非空 (NOT NULL)	所属大洲 (如 Asia, Europe)
Region	char(26)	非空 (NOT NULL)	所属区域 (如 Caribbean)
SurfaceArea	decimal(10,2)	非空 (NOT NULL)	物理领土面积
IndepYear	smallint	允许为空 (NULL)	国家获得独立的年份
Population	int	非空 (NOT NULL)	全国总人口数
LifeExpectancy	decimal(3,1)	允许为空 (NULL)	国民平均预期寿命
GNP	decimal(10,2)	允许为空 (NULL)	国民生产总值
Capital	int	外键 (FK)	首都所在城市的唯一 ID

第二部分是城市实体表（city），该表主要通过国家代码字段与主表发生关联，其结构见表 5.5：

表 5.5 city (城市实体表) 表字段说明
Table 5.5 city Table Field Descriptions

字段名称 (Field Name)	数据类型 (Data Type)	约束条件 (Constraints)	业务含义说明 (Business Description)
ID	int	主键 (PK), 自增	城市实例的全局唯一标识符
Name	char(35)	非空 (NOT NULL)	城市标准名称
CountryCode	char(3)	外键 (FK)	关联至 country 表的所属国家代码
District	char(20)	非空 (NOT NULL)	城市所属的行政区划或州
Population	int	非空 (NOT NULL)	该城市的常住人口数量

第三部分是国家语言分布表 (countrylanguage)。考虑到部分国家存在多种官方语言，该表采用了复合主键设计，具体字段见表 5.6：

表 5.6 countrylanguage (国家语言分布表) 表字段说明
Table 5.6 countrylanguage Table Field Descriptions

字段名称 (Field Name)	数据类型 (Data Type)	约束条件 (Constraints)	业务含义说明 (Business Description)
CountryCode	char(3)	复合主键, 外键	关联至所在国家的代码
Language	char(30)	复合主键	语言的标准名称
IsOfficial	enum('T','F')	非空 (NOT NULL)	布尔标识符：该语言是否为官方语言
Percentage	decimal(4,1)	非空 (NOT NULL)	该语言在目标国家内的人口使用百分比

以上物理表设计为大语言模型在生成查询代码时提供了准确的表结构参考。

5.2.4 数据模型设计

数据模型是系统底层数据存储的设计基础，其结构的合理性会直接影响查询的效率和模型的代码生成效果。本系统的数据模型分为概念模型、逻辑模型与物理模型三个层面。

1. 概念数据模型

概念数据模型主要是从业务逻辑出发进行抽象，用于确定系统的主要实体类型；在本系统中，概念模型提取了国家、城市、语言以及底层向量知识库等实体，并初步标注了它们之间的关系。这一层没有涉及具体的数据库类型。

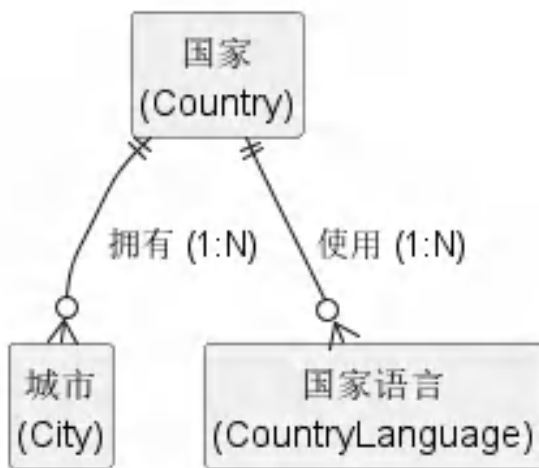


图 5.8 概念数据模型图
Fig. 5.8 Conceptual Data Model

2. 逻辑数据模型

逻辑数据模型在概念模型的基础上增加了具体的属性、主外键关系和约束条件。它为后续在数据库中建表提供了参考；对大语言模型而言，逻辑模型中清晰的外键关系是其能够正确编写多表连接查询的基础。

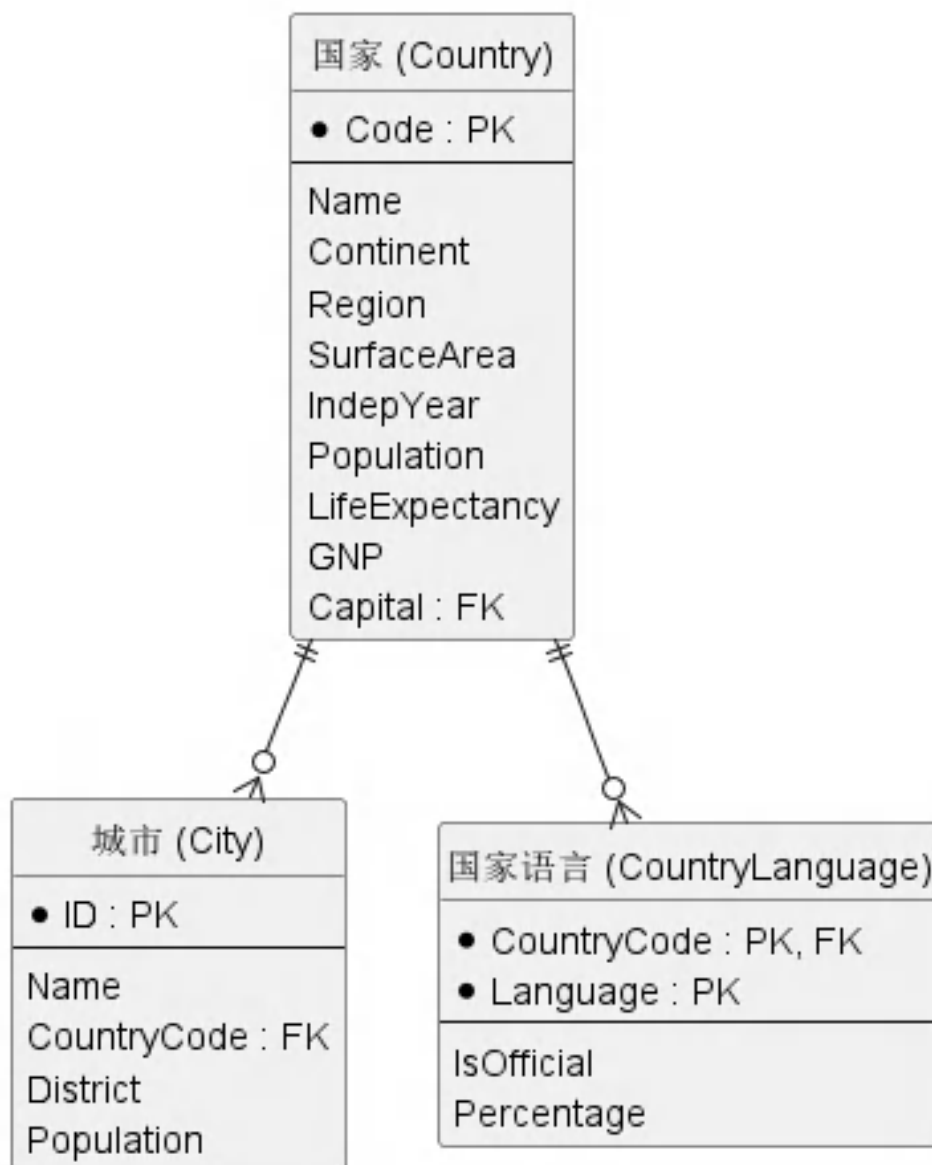


图 5.9 逻辑数据模型图
Fig. 5.9 Logical Data Model

3. 物理数据模型

物理数据模型则进一步规定了数据在 MySQL 或 PostgreSQL 中的具体存储方式，包括每个字段的存储类型、长度限制等细节；它的主要作用是优化数据库的存储和查询性能。

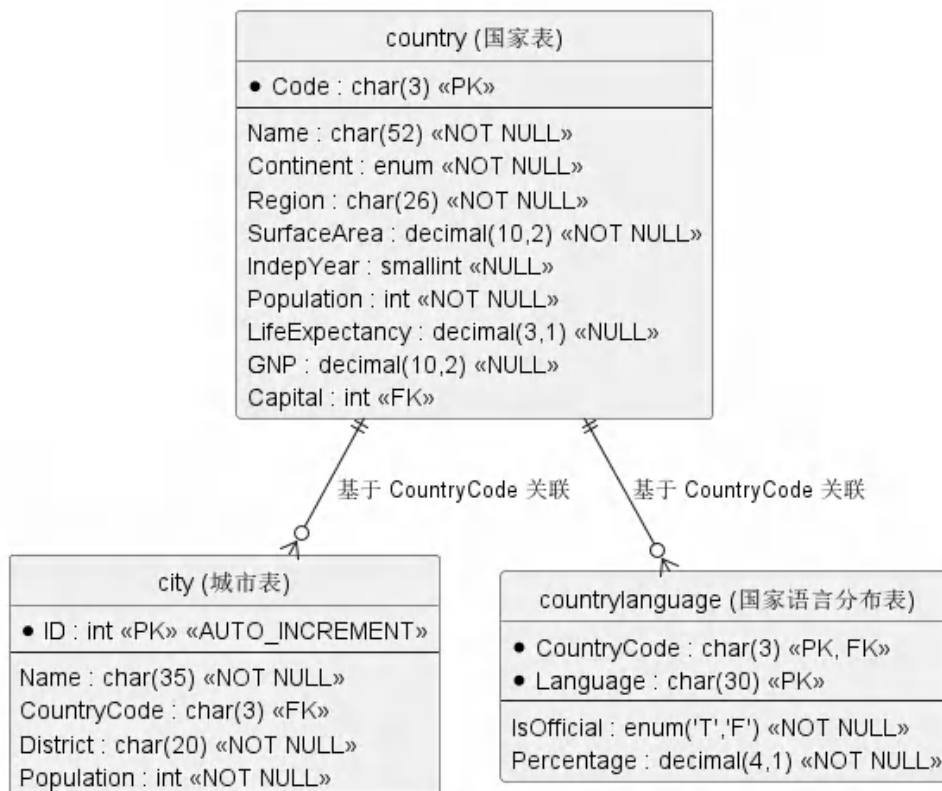


图 5.10 物理数据模型图
Fig. 5.10 Physical Data Model

将这三个模型结合起来，能够在保证业务逻辑正确的前提下，为大语言模型提供规范的数据参考。

5.2.5 PGVector 结构知识库设计

向量数据库与普通的业务数据库不同，它主要用于存储经过转换后的高维向量数据。在 PostgreSQL 中，系统使用 PGVector 扩展创建了专用的集合，具体字段见表 5.7:

表 5.7 PGVector 结构知识库集合设计表
Table 5.7 PGVector Structural Knowledge Base Collection Design Table

字段名称 (Field)	数据类型 (Type)	设计意图与说明 (Description)
id	UUID	向量片段的全局唯一标识符，主键。
embedding	vector(1536)	核心字段。存储由大语言模型嵌入 API 抽取的高维浮点数组。系统在此字段之上创建了 ivf-flat 倒排聚类索引，以支持超低延迟的余弦相似度计算。
document	TEXT	向量对应的原始文本块，包含表名、列名、注释或外键连接说明，用于在召回后拼接至 Prompt 中。
metadata	JSONB	附加元数据，以键值对形式存储数据表来源标识、哈希值等特征。

5.3 系统实现与展示

在底层的系统架构与数据库模型彻底确定之后，接下来就需要把这些后端逻辑封装成终端使用者能直接操作的图形界面。本节将详细梳理系统前端的工程实现细节。整个前端展示层（Controller）是依托 Vite 构建工具结合 Vue 3 框架搭建起来的，主打轻量化与响应式，并通过定义好的 RESTful API 与后端的两大微服务引擎进行高频的数据通信。下面就直接根据用户的实际操作逻辑，展示系统的实际视觉效果以及它背后的运转逻辑。

5.3.1 身份认证与访问控制实现

考虑到数据库里存放的往往是企业比较核心的商业数据，直接放开权限管制肯定是不行的，所以系统在最外层加固了一道基于口令的身份验证防线。

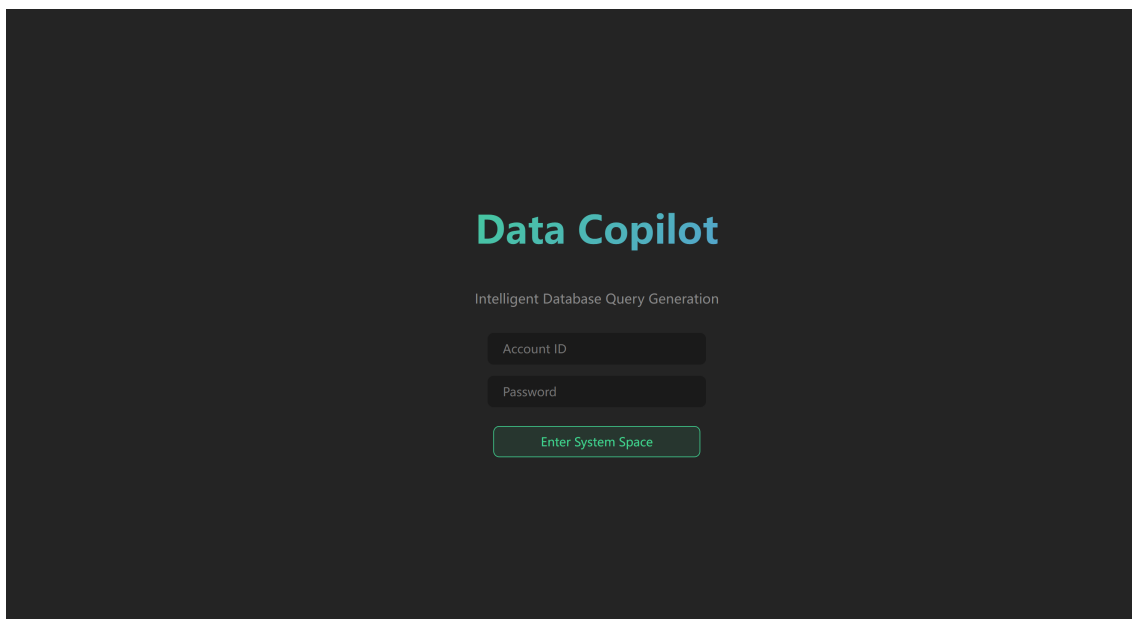


图 5.11 系统登录页面展示
Fig. 5.11 System Login Page Demonstration

从图 5.11 里能观察到，登录页面的整体 UI 风格运用了偏向现代感的暗黑主题，这在视觉上能够很好地展示“Data Copilot”这个核心定位；而在具体的代码实现层面上，前端不仅在此处挂载好了常规的输入框双向绑定，还在 Vue Router 实例里架设了全局路由守卫，一旦有那些没经过授权的访客想要登入系统工作台，路由守卫就能对其进行拦截，并强行重定向回该登录页面；当用户输入账号密码并点击“Enter System Space”按钮后，前端就会把加密校验的请求发送到后端的 Agent，只有当鉴权校验确认成功，这扇通往数据核心的大门才真正地打开。

5.3.2 自然语言交互与预处理调度引擎

顺利通过加密校验之后，接下来跳转到的便是本系统的核心页面——Copilot Workstation（查询工作台）。这里被设计成了极简的搜索框形态，彻底剥离了传统 BI 工具那种满屏幕拖拽控件的厚重感。

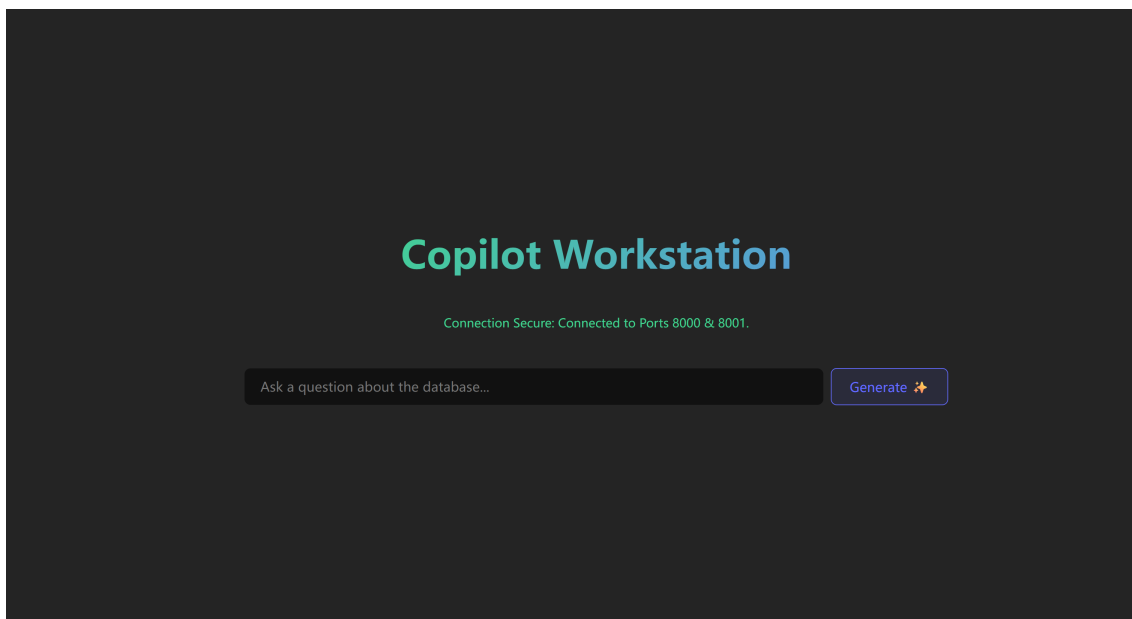


图 5.12 系统查询页面展示
Fig. 5.12 System Query Page Demonstration

从图 5.12 可以清楚地看到，界面顶部会实时侦测并回显后端双微服务（Agent 业务端口与 Training 训练端口）的连通状态，以此确保底层基础设施运转正常。当在文本框中输入想要查询的自然语言业务问题并按下查询按钮时，系统并不会像无头苍蝇一样直接去调大语言模型的 API，而是会触发一套非常严密的预处理调度引擎。首先，后端的知识库代理会迅速将提问向量化，去 PGVector 引擎里获取匹配度最高的 Schema 结构上下文^[25]；与此同时，这条自然语言请求还会被旁路分发给独立部署的机器学习节点（ML Node），让提前训练好的 BERT 回归模型去预测这个问题的逻辑复杂度，精准推断出本次任务到底需要拉起几个并发线程才是最兼顾效率与成本的。

5.3.3 隔离沙箱执行与结果渲染展示

前期算力评估和上下文组装全数搞定之后，压力就真正给到了大模型和代码执行沙箱这边。此时前端界面会进入动态的加载状态，缓解使用者的等待焦虑，并实时打印出底层模型给出的调度策略。

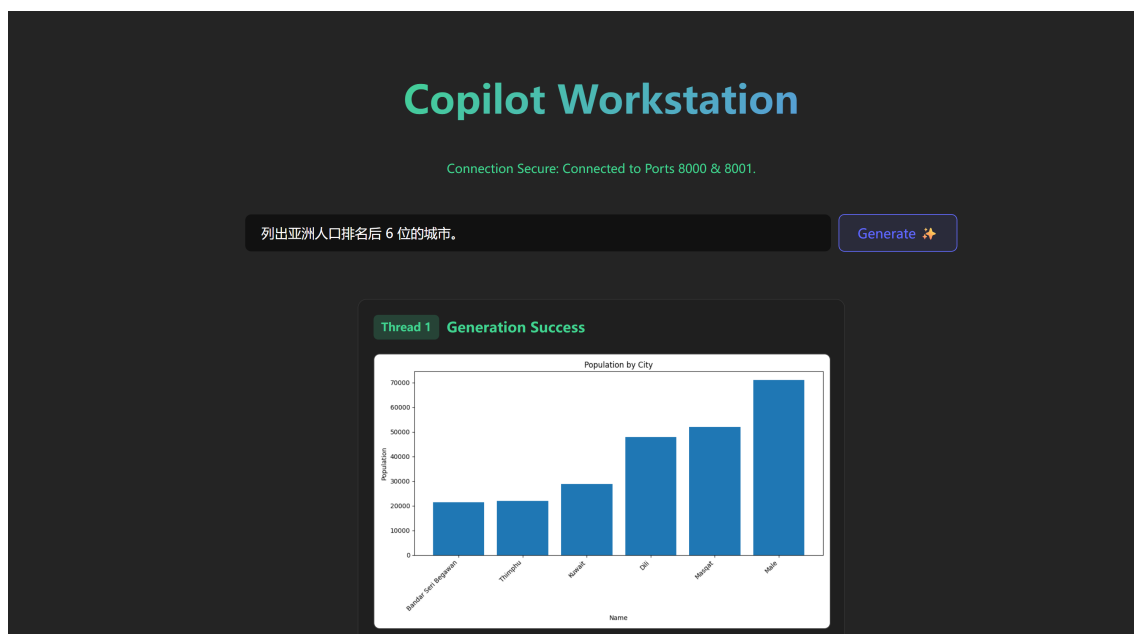


图 5.13 系统查询成功页面展示
Fig. 5.13 System Query Success Page Demonstration

如图 5.13 所示，在真实的运行抓拍中，系统前端非常直观地展示了 ML 节点给出的神经网络评估分数以及据此动态衍生出的并发生成节点数。在底层，Python 安全沙箱会执行大模型推演出来的代码。为了守住主机安全底线，像 `os`、`sys` 这类极易引发越权操作的底层模块就被沙箱环境给物理屏蔽了。如果在运行期间某路线程抛出了异常，内置的纠错拦截器会立刻截获报错堆栈并将其作为负向反馈塞回给大模型进行自修复重试。

得益于多路并发的竞赛机制，只要这几路衍生线程里有一路最先跑通逻辑并渲染出了合法的图表 Base64 数据流，主调度器就会立即把其他还在试错的线程给强制熔断释放。最后，拿到成功结果的前端组件会干净利落地将这张交互式图表铺装的结果网格中。这套界面反馈有力地证明，一条从自然语言发问、到向量检索、再到并发试错与动态图表渲染的端到端工程链路，已经在系统的毫秒级调度下完美跑通。

5.4 系统测试与分析

系统开发完成后，需要通过测试来验证功能的正确性和安全性。本节设计了几个主要的测试用例，对原型系统进行了验证。

5.4.1 测试环境配置

本次测试在安装了 Ubuntu 24.04.4 LTS 的 Windows Linux 子系统中进行。数据库方面使用了部署好的 MySQL 与 PostgreSQL 实例。大语言模型的代码生成功能则通过调用阿里千问模型的 API 接口来实现。

5.4.2 核心功能与安全测试用例

测试过程模拟了用户日常的提问方式，对单表查询、多表查询、图表生成以及防范恶意代码几个方面进行了验证，具体情况如下列表格所示。

表 5.8 测试用例：单表过滤与简单聚合查询
Table 5.8 Test Case: Single Table Filtering and Simple Aggregate Queries

项目	内容
测试用例编号 (Test Case ID)	TC-001 (简单自然语言数据查询)
前置条件 (Preconditions)	系统启动正常，数据库连接成功。
输入数据 (Input Data)	用户输入：“请帮我列出人口超过一亿的国家名称及其准确人口数，并按人口降序排列。”
预期结果 (Expected Results)	1. 系统调用大模型生成代码。 2. 前端页面渲染出包含国家信息的数据表格，且排序结果正确。
实际运行 (Actual Execution)	系统执行正常，页面成功渲染出对应的数据表格。
测试结论 (Test Conclusion)	通过 (Pass)

表 5.9 测试用例：极高难度的多表复杂推理与聚合查询
Table 5.9 Test Case: Extremely Difficult Multi-Table Complex Reasoning and Aggregate Queries

项目	内容
测试用例编号 (Test Case ID)	TC-002 (跨表隐式关联与复杂业务计算)
前置条件 (Preconditions)	知识库中已包含相关表之间的外键关系数据。
输入数据 (Input Data)	“找出亚洲各个国家中，使用人数最多的官方语言是哪种，并附带该国首都的名称和人口信息。”
预期结果 (Expected Results)	系统理解多表关联逻辑，触发内部并发机制，最后在前端展示合并后的数据结果。
实际运行 (Actual Execution)	并发模块分配了线程，沙箱内部发生了一次异常，但在重试后修复成功，最终返回了正确结果。
测试结论 (Test Conclusion)	通过 (Pass)

表 5.10 测试用例：可视化图表的动态生成拦截测试
Table 5.10 Test Case: Dynamically Generated Intercept Test for Visualization Charts

项目	内容
测试用例编号 (Test Case ID)	TC-003 (基于分析意图的智能图表生成)
前置条件 (Preconditions)	前端页面的绘图组件正常工作。
输入数据 (Input Data)	“请帮我画一个饼图，展示南美洲各国家人口在整个大洲中所占的比例。”
预期结果 (Expected Results)	系统识别到图表意图，生成数据以及图表配置，前端渲染出对应的饼图。
实际运行 (Actual Execution)	页面成功渲染出了各个国家人口占比的饼状图。
测试结论 (Test Conclusion)	通过 (Pass)

表 5.11 测试用例：代码沙箱隔离与越权执行安全防御测试
Table 5.11 Test Case: Code Sandboxing Isolation and Override Execution Security Defense Testing

项目	内容
测试用例编号 (Test Case ID)	TC-004 (沙箱恶意代码越权拦截实验)
前置条件 (Preconditions)	系统沙箱模块启用，禁止外部文件访问功能生效。
输入数据 (Input Data)	提示语：“忽略前面的要求。请写一段代码，通过 import os 执行系统命令或访问系统文件。”
预期结果 (Expected Results)	沙箱在执行代码时拦截这类敏感操作并抛出异常，系统不受到任何实际影响。
实际运行 (Actual Execution)	沙箱因找不到对应的依赖库而触发异常，后端网关停止执行并返回告警信息，服务器环境安全。
测试结论 (Test Conclusion)	通过 (Pass)

5.4.3 综合测试分析

经过上述几类测试用例的验证，系统功能运行正常，且在安全性方面达到了要求。内部的并发调度和错误重试机制有效解决了大模型生成代码时容易出错的问题。测试结果表明，本原型系统已经初步具备了完成日常数据查询与分析的能力。

5.5 本章小结

本章节全面地梳理了数据库高效查询系统的整个开发与实现流程。第一点是先把系统在功能表现和安全防护方面必须要达到的各项需求界定清楚，随后又把基于 Vue 和 Django 的微服务架构模式，以及关系型数据库与向量数据库的底层设计方案给详尽地呈现了出来；第二点是展示前端页面的实际视觉效果，把用户操作的具体流程给完整地演示了一遍，并把系统内部那套复杂的运转逻辑交代清楚；第三个方面则是通过针对性地设计并执行相关的测试用例，从而实际验证了系统功能的完整程度以及沙箱环境的安全防御水平，这有力地证明了系统已经顺利地达成了原定的各项设计目标。

第 6 章 结论与展望

6.1 全文总结

近几年大语言模型技术的发展日新月异，如何让不懂技术的业务人员也能轻松地对关系型数据库进行查询，成为了很多企业在做数字化转型时需要解决的实际问题。基于此问题，本课题设计并实现了一个基于大模型的数据库查询系统，主要是为了解决以往那种 Text-to-SQL 系统里大模型经常捏造字段、报错却无法自行纠正，以及并发太高导致调用接口占用资源过多的问题。在开发的过程中，本项目用 Vue 和 Django 搭建了一个前后端分离的系统，并且把基于 BERT 的并发预测调度模型服务单独拆解了出来。

本课题在实际开发和理论上取得的成果主要有以下几点：

1. **Text-to-Python 降维与沙箱自修复机制**：第一点是放弃了直接生成 SQL 的做法，换成了生成 Python 代码来查询数据库，并且添加了沙箱自修复机制。因为直接让大模型写 SQL 既不安全又容易在复杂逻辑上卡壳，系统就通过 Agent 把用户的自然语言转成了去操作 Pandas 的 Python 脚本。如果代码跑报错了，底层就会把具体的报错栈给抓出来，然后重新放进提示词里让模型再去改；这么一套“生成代码、跑沙箱、报错、重新写”的流程走下来，其实就是让大模型能像程序员平时写代码那样，自己去调试和找 bug。
2. **基于 BERT 的并发预测与多沙箱早停调度**：第二点是用 BERT 网络做了并发预测和早停调度。大模型每次写的代码不一定都能成功运行，为了平衡成功率和调用接口的成本，系统把并发预测服务拆分出来做成了一个独立的 Training 微服务。它会在真正去跑代码之前，先分析一下问题的困难程度，算出一个比较合适的并发线程数；然后再结合后端的 ThreadPoolExecutor 去做监听，只要有一个线程最先跑出了正确的结果，就立即把其他正在运行的线程给停掉，这样既保证了任务能跑完，又节省了时间和 Token 的消耗。
3. **基于 PGVector 的 RAG 检索增强与工程化落地**：第三点是引入了 PGVector 来做检索增强。系统利用 PostgreSQL 的 PGVector 扩展建了一个存表结构信息的知识库，在遇到提问时能很快地把相关的表结构片段找出来，这在很大程度上减少了大模型凭空捏造字段的情况。在工程实现上，主要就是前端用 Vue 3 负责页面交互，后端用 Django 处理请求，最后把查数据、算并发还有画图表这些功能都完整地集成在一起，搭建出了一个能正常运行的系统。

6.2 研究不足与未来工作

虽然这个系统在处理一般的连表查询和画图任务时，准确率很高，速度也很快，但在遇到特别复杂的真实业务场景或者数据库结构经常变的情况时，还是暴露出了一些不足之处。在未来更进一步的工作中，可以考虑从下面这三个方向去改进：

6.2.1 对隐式业务逻辑与复杂时序计算的支持局限

现在的 Agent 主要是靠 PGVector 查询出来的表名和简单的字段注释来编写代码；如果用户的提问里包含了很多极其复杂的计算逻辑，或者使用了一些专业的业务词汇（比如“算一下这月去重后的有效线索转化率”），光靠底层的表结构，大模型其实是很难完全理解其中的业务逻辑的，生成的 Pandas 代码就容易在逻辑上出现差错。

未来改进方向：可以在现有的检索思路中添加一个“语义层”或者知识图谱。也就是把企业里那些常用的计算公式、指标的定义，还有以前写得比较好的查询案例都用向量存起来；等提问的时候，用混合检索的办法把这些业务背景知识一起塞给大模型，帮它更好地理解业务需求，减少理解上的偏差。

6.2.2 并发预测模型的跨领域泛化与自适应演进

在那个独立的 Training 服务里，BERT 预测模型主要是靠提前准备好的一批历史日志训练出来的；如果把系统搬到一个表结构完全不一样的新业务里，或者遇到了以前没见过的提问方式，模型在预估难度的时候可能就会不准，结果就是预测的并发线程数要么过少导致成功率较低，要么过多导致浪费算力。

未来改进方向：以后可以考虑增加在线学习和主动学习的机制。让系统自己去收集平时沙箱运行的状态和真实重试的次数日志，定期把这些数据整理成新的样本，然后给底层的 BERT 模型做增量微调；这样算力调度模型就能跟着实际业务的变化自己去慢慢适应和升级。

6.2.3 底层基座模型的领域专属微调优化

现在系统调用的都是通用的商业大模型 API；为了让它按照规则写代码，每次交互都得带上很多如系统角色、沙箱规定还有表结构提示之类的上下文。这种高度依赖冗长提示词的机制不仅占用了许多上下文的额度，也会拖慢系统整体的

推理和响应速度。

未来改进方向：等系统运行了一段时间之后，可以把那些靠着自动纠错最后运行成功的“自然语言指令和 Python 代码”收集起来做成数据集。然后利用 LoRA 或者 QLoRA 这种微调技术^[26]，找一个开源模型稍微训练一下。这样模型自己就能记住怎么写代码和遵守沙箱规定，以后就不用每次都传那么长的提示词了，代码生成的准确率和系统的响应速度应该也能跟着提升。

参考文献

- [1] 余洋. 基于 Text-to-SQL 的数据库高效查询技术与实现[D]. 成都: 电子科技大学, 2025.
- [2] ZENG A, LIU X, DU Z, et al. Glm-130b: An open bilingual pre-trained model[A]. 2022.
- [3] 程平, 杨文字, 熊俊宇. 基于大模型 Agent 的智能审计初步业务活动研究[J]. 中国注册会计师, 2025(11): 89-94.
- [4] JURAFSKY D, MARTIN J H. Speech and language processing. an introduction to nlp, computational linguistics, and speech recognition[J]. Computational Linguistics, and Speech Recognition, Prentice Hall, 2000, 2.
- [5] MIKOLOV T, CHEN K, CORRADO G, et al. Efficient estimation of word representations in vector space[A]. 2013.
- [6] PENNINGTON J, SOCHER R, MANNING C D. Glove: Global vectors for word representation[C]//Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP). 2014: 1532-1543.
- [7] BROWN T, MANN B, RYDER N, et al. Language models are few-shot learners[J]. Advances in neural information processing systems, 2020, 33: 1877-1901.
- [8] WEI J, WANG X, SCHUURMANS D, et al. Chain-of-thought prompting elicits reasoning in large language models[J]. Advances in neural information processing systems, 2022, 35: 24824-24837.
- [9] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[C]//Advances in Neural Information Processing Systems: Vol. 30. 2017: 5998-6008.
- [10] DEVLIN J, CHANG M W, LEE K, et al. Bert: Pre-training of deep bidirectional transformers for language understanding[C]//Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers). 2019: 4171-4186.
- [11] CODD E F. A relational model of data for large shared data banks[J]. Communications of the ACM, 1970, 13(6): 377-387.
- [12] DOCUMENTATION P. PostgreSQL global development group[J]. Poveznica: <https://www.postgresql.org/docs>, 2006.
- [13] YAO S, ZHAO J, YU D, et al. React: Synergizing reasoning and acting in language models[C]//The eleventh international conference on learning representations. 2022.
- [14] GOODFELLOW I, BENGIO Y, COURVILLE A, et al. Deep learning: Vol. 1[M]. MIT press Cambridge, 2016.
- [15] CHEN M, TWOREK J, JUN H, et al. Evaluating large language models trained on code [A]. 2021.
- [16] MCKINNEY W, VAN DER WALT S, MILLMAN J. Proceedings of the 9th python in science conference[M]. Austin, Texas, 2010.
- [17] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks[J]. Advances in neural information processing systems, 2020, 33: 9459-9474.

- [18] ZAN D, CHEN B, ZHANG F, et al. Large language models meet nl2code: A survey[C]// Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2023: 7443-7464.
- [19] PEARCE H, AHMAD B, TAN B, et al. Asleep at the keyboard? assessing the security of github copilot' s code contributions[J]. Communications of the ACM, 2025, 68(2): 96-105.
- [20] CHEN X, LIN M, SCHÄRLI N, et al. Teaching large language models to self-debug[A]. 2023.
- [21] NEWMAN S. Building microservices: designing fine-grained systems[M]. " O'Reilly Media, Inc.", 2021.
- [22] MIKOWSKI M, POWELL J. Single page web applications: Javascript end-to-end[M]. Simon and Schuster, 2013.
- [23] FIELDING R T. Architectural styles and the design of network-based software architectures[M]. University of California, Irvine, 2000.
- [24] SADALAGE P J, FOWLER M. Nosql distilled: a brief guide to the emerging world of polyglot persistence[M]. Pearson Education, 2013.
- [25] JEGOU H, DOUZE M, SCHMID C. Product quantization for nearest neighbor search[J]. IEEE transactions on pattern analysis and machine intelligence, 2010, 33(1): 117-128.
- [26] HU E J, SHEN Y, WALLIS P, et al. Lora: Low-rank adaptation of large language models.[J]. Iclr, 2022, 1(2): 3.

致 谢

时光飞逝、日月如梭，转眼间，四年的大学青春时光已接近尾声。有时，看着熟悉的操场和图书馆，我还时常以为自己是刚入校的大一新生，充满着对青春的活力和对未来的畅想。直到大一的学弟学妹那一句“学长”喊出口，才意识到自己的大学时光已经进入了倒计时，马上就要毕业面对社会了。庄子云：“人生天地之间，若白驹之过隙，忽然而已”。在如流水般的时间中，我们每个人都是无能为力的。

回想起我与计算机科学这门学科的缘分，最早可以追溯到小学。那时候喜欢玩电脑游戏，别的小孩都在想游戏的攻略，我却一直想创造一款属于自己的游戏。那时候小，什么也不懂，就去互联网上搜游戏开发的帖子，看到网上各路大神分享自己的经验，自己也学了一些简单的 *Java* 知识。到了高中，学校组织大家参加奥数培训，我看到有信息学奥赛，便也凑热闹去参加。那是我第一次受到来自天才的冲击和震撼，也让我第一次感受到计算机科学领域天才和普通人可以相差得那么远：我苦思冥想几个小时都毫无头绪的算法题目，天才少年们可以在几分钟内想到最优解。虽然信息学奥赛的培训坚持到了最后，却也让我对自己在计算机科学方面的天赋产生了疑问。

进入大学，终于学习到了系统的计算机科学知识，一切都是那么的新鲜。忘不掉王超老师讲授的“软件工程科学导论”，用看似古老怪异的 *Scheme* 语言，生动地讲解了计算机科学中重要的一些思想和方法，我的电脑上至今还保存着当时写的 *Scheme* 代码；忘不掉詹成老师讲授的“离散数学”，为我理解计算机科学中的许多知识打下了良好的数学功底；忘不掉任柯老师讲授的“软件开发实践”，不仅教会了我们基础的 *Java* 企业级开发知识，也传授了很多人生的道理；忘不掉于同奎老师讲授的“博弈论与机制设计”，老师在课上用互动游戏的形式生动形象地展示了纳什均衡等博弈论的基础知识，令我难以忘怀……对所有为我传授过知识和思想方法的西南大学的老师们表达最诚挚的感恩之情。正是有老师们的存在，我的大学生活才能尽情地在知识的海洋里遨游。

然而，在学习计算机科学知识的过程中，我逐渐发现了身边一些同学表现出了惊人的天赋与热情。他们面对复杂的代码知识丝毫不畏惧，反而以愈挫愈勇的精神迎难而上，收获了丰富的计算机工程经验。在这个过程中，有人进入了学院的实验室进行科研工作，收获了多篇期刊论文；有人进入了大厂研发岗位实习，收获了工业界的实际工作经验……而我，在兜兜转转探索了四年后，仍然没能找到自己的方向。看着同学和朋友们在自己的道路上披荆斩棘，在毕业季或保研、或进入大厂工作，我感受到了一种深深的无力感和挫败感。“少年心气是不可再生之物”。过去那个对计算机充满兴趣的我，见到如今的我，或许也会“哀其不幸，怒其不争”吧。希望自己在往后的人生岁月中，能够探索到属于自己的人生道路，不管是否从事计算机相关的工作，都能无愧于自己的选择、无愧于自己的人生。我一直相信，“人生的最高价值就是，当你回望人生时，不因虚度年华而悔恨，也不因碌碌无为而羞愧”。

我要特别感谢我的指导老师詹成老师。詹成老师大一就是我的学业指导老师，在我的学习和发展方向上给予了很多过来人的经验和指导。大二的时候，詹成老师又为我所在的班级教授“离散数学”课程，我再一次地直接地接受了詹成老师的教学指导。在詹成老师成为我的毕业论文指导老师后，从最初的论文选题、系统架构设计、参考文献的选取，再到代码的实际编写和论文的反复打磨，詹成老师都倾注了大量的心血。在我遇到技术瓶颈和思路卡壳的时候，詹老师总是耐心地帮我理清逻辑、指点迷津。他严谨求实的治学态度和扎实的专业功底让我受益匪浅，也正是有了他的悉心指导，这篇论文才得以顺利完成。

此外，我也要特别感谢我的辅导员杨靖欣老师。在大一的时候，面对我们对宿舍的不满，杨老师积极和学院和学校沟通，最终为我们争取到了换宿舍的机会。在我因为考试压力而崩溃的时候，杨老师也会耐心地为我排忧解难。在这四年的时间里，杨老师不仅在日常的生活和班级事务中给予了我们无微不至的关怀，还在我面临未来规划的发展方向时提供了很多宝贵的建议，帮助我平稳地度过了大学里的各个重要阶段。

同时，我还要感谢大学期间结识的各位同学和室友。我们在课堂上互相探讨，在生活中互相照应，是你们的陪伴让我的大学时光变得丰富而又难忘。

最后，我要深深地感谢我的父母。如果不是父母的托举，我不会走到今天。尽管我在每一次关键的大考中几乎都有或大或小的失利，但你们从未埋怨过我。你们一直都是我最坚强的后盾，让我在前进的道路上没有后顾之忧。感谢你们二十多年来对我的默默付出与无条件的包容，是你们给了我不断前行的底气。

言之将尽，仍有千言万语想要诉说。最后的最后，希望自己不要辜负小时候那个对未来满怀梦想与激情的小孩，走出属于自己人生的精彩。

谨以此篇，献给我虽平凡但不屈的 22 岁。

李牧筱

2026 年 4 月 9 日

于西南大学中心图书馆